Singularity-Encoded Green's Functions for Iterative Methods

Qi Sun

School of Mathematical Sciences, Tongji University



同济大学数学科学学院

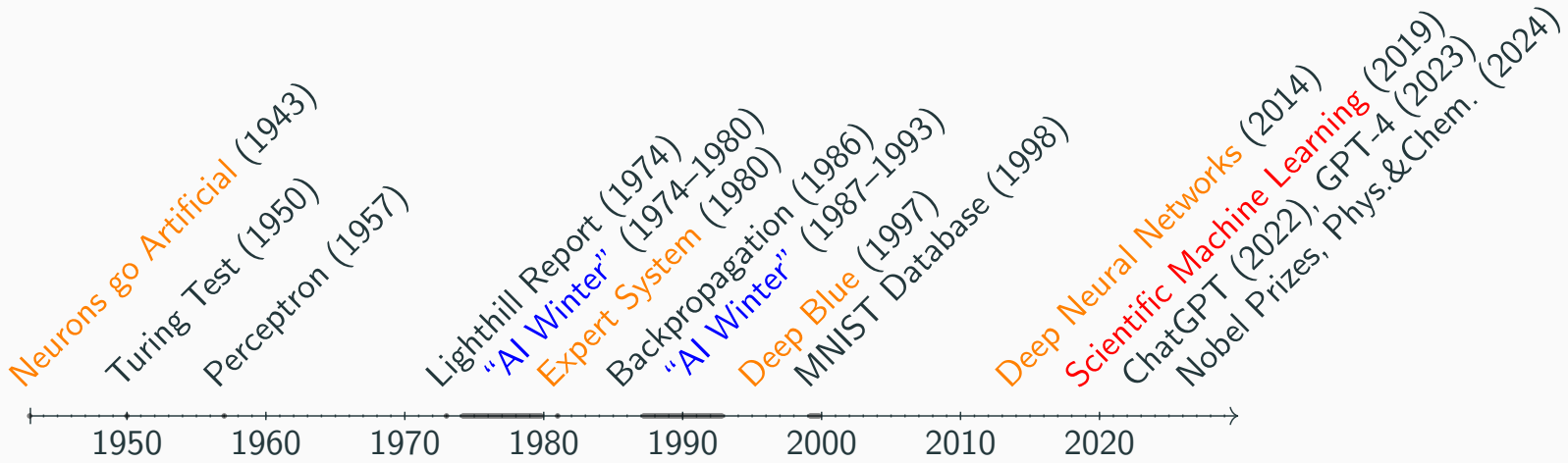
School of Mathematical Sciences, Tongji University

Outline

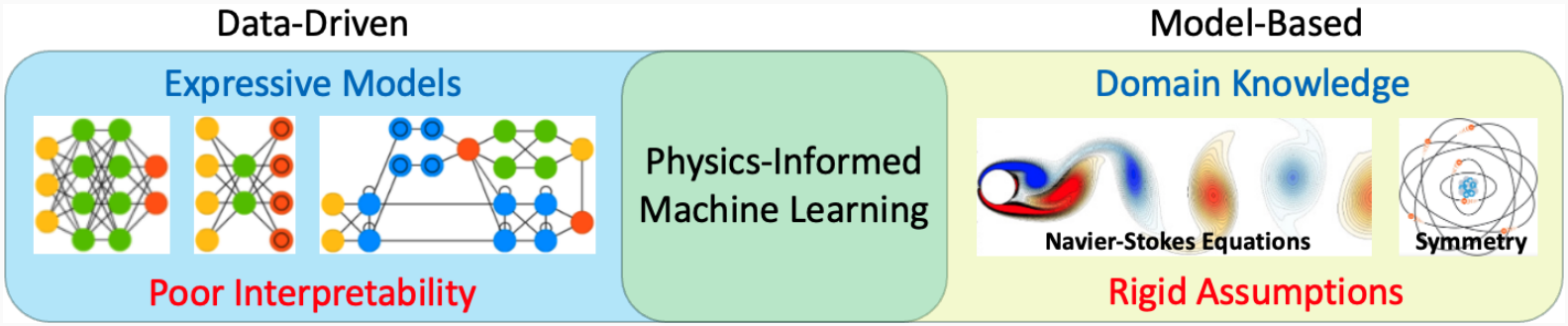
1. **Background**
2. **Learning Singularity-Encoded Green's Function**
3. **Applications in Accelerating Iterative Methods**
4. **Concluding Remarks**

1. Background

The Evolution of Artificial Intelligence

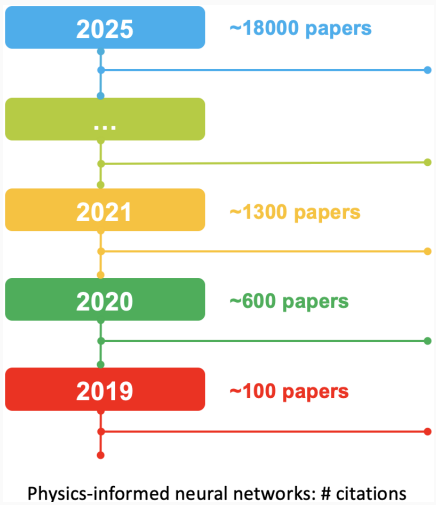


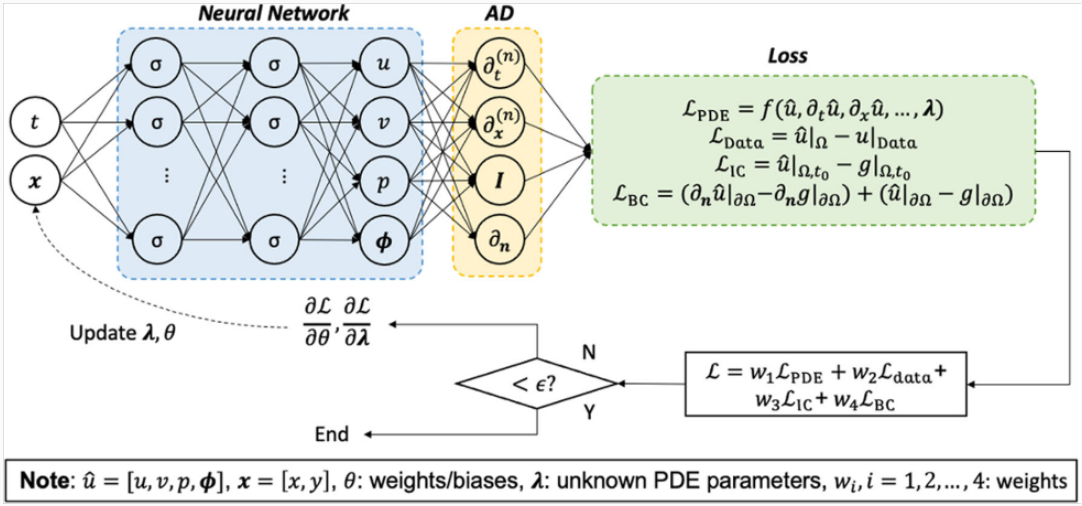
SciML is a new discipline that blends scientific computing and machine learning.



§S. Brunton et al., Machine learning for fluid mechanics, 2020
§G. Karniadakis et al., Physics-informed machine learning, 2021
§S. Cuomo et al., SciML through PINNs: Where we are and what's next, 2022
§S. Brunton et al., Promising directions of machine learning for PDEs, 2023
§H. Wang et al., Scientific discovery in the age of artificial intelligence, 2023
§T. Ryck et al., Numerical analysis of PINNs and models in physics-informed machine learning, 2024
§K. Azizzadenesheli et al., Neural operators for accelerating scientific simulations and design, 2024

Function Learning

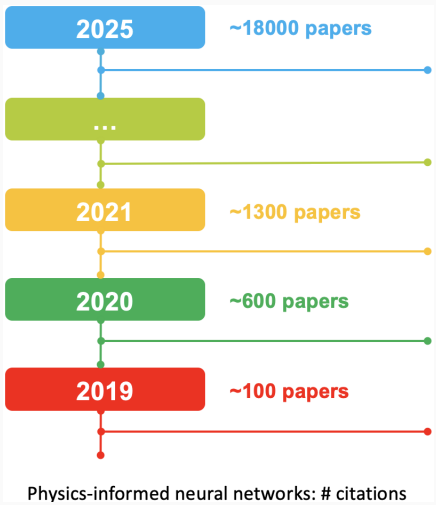


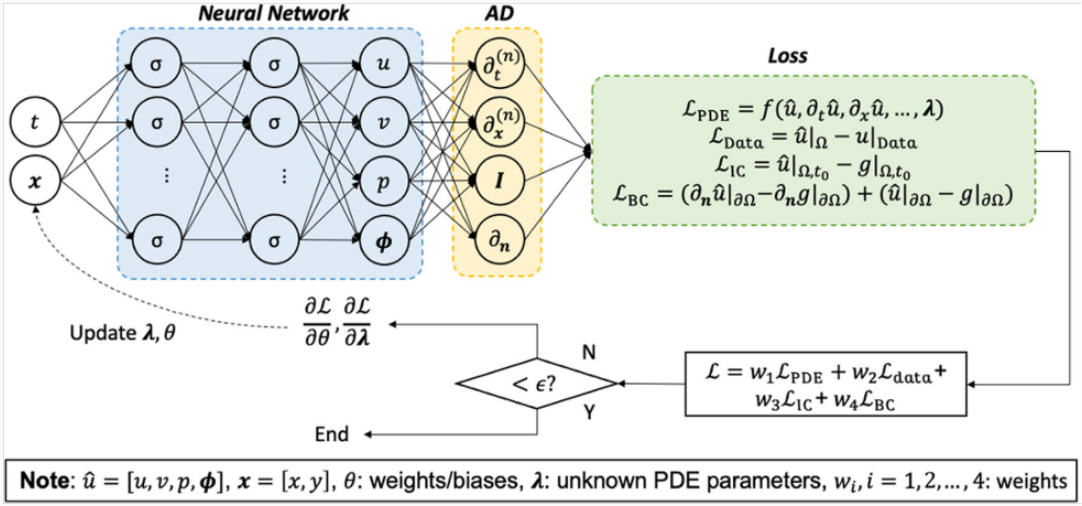


[§] I. Lagaris et al., Neural-network methods for boundary value problems with irregular boundaries, 2000

[§] M. Raissi et al., PINNs: DL for solving forward/inverse problems involving nonlinear PDEs, 2019

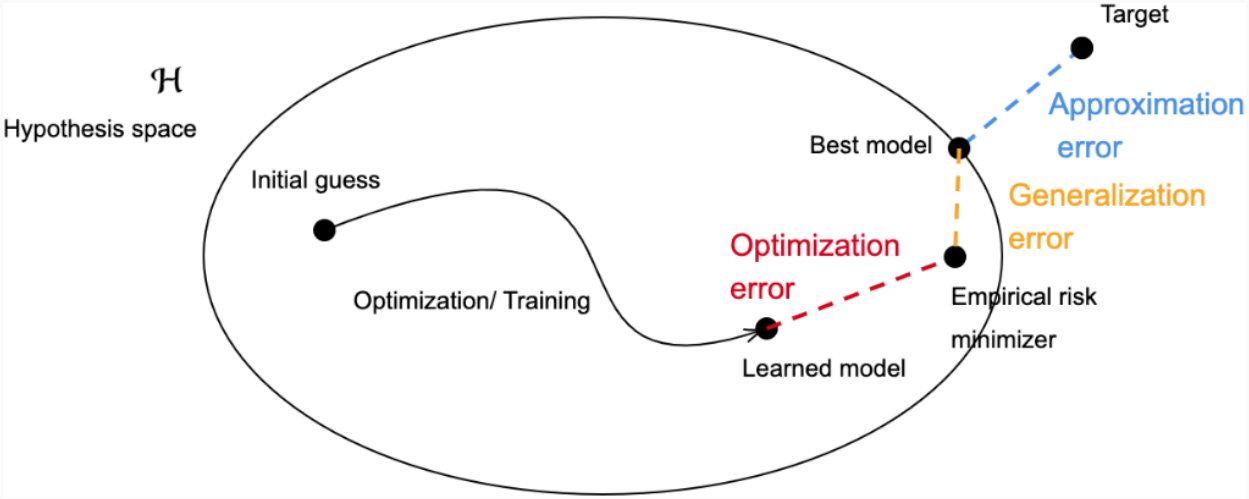
Function Learning



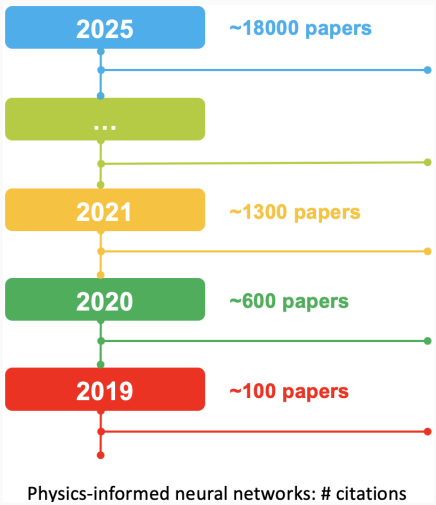


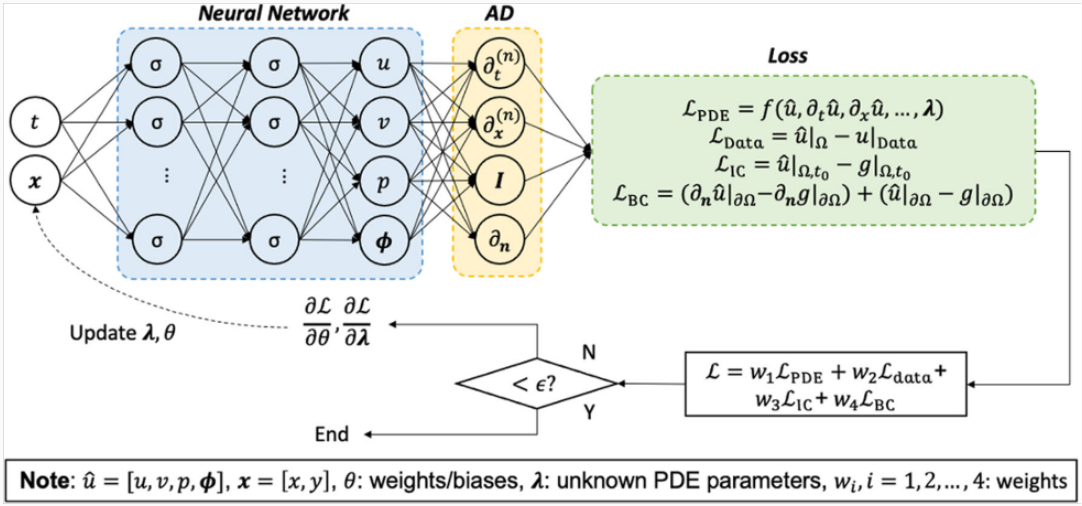
[§] I. Lagaris et al., Neural-network methods for boundary value problems with irregular boundaries, 2000

[§] M. Raissi et al., PINNs: DL for solving forward/inverse problems involving nonlinear PDEs, 2019



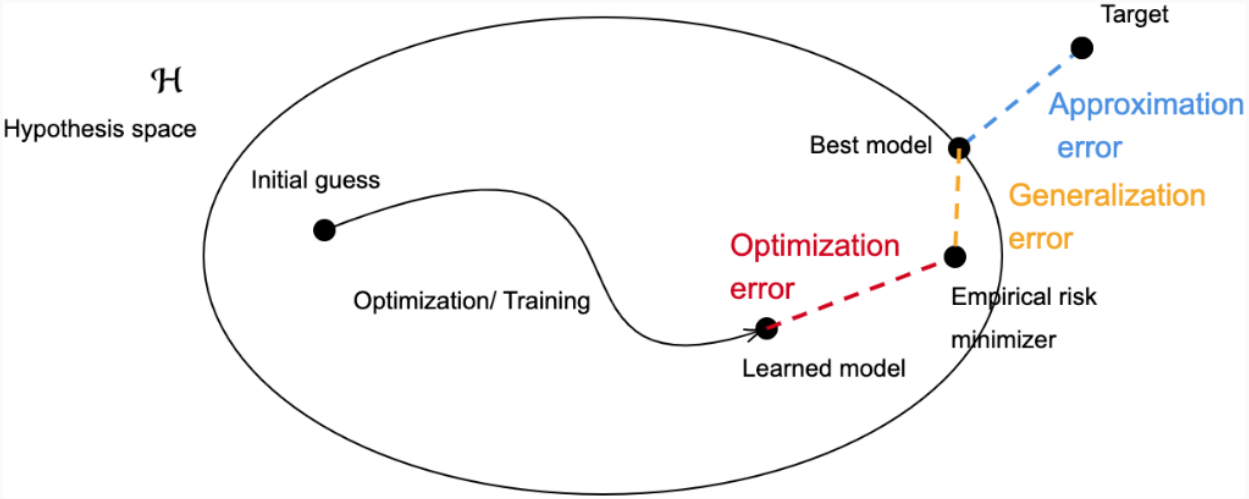
Function Learning





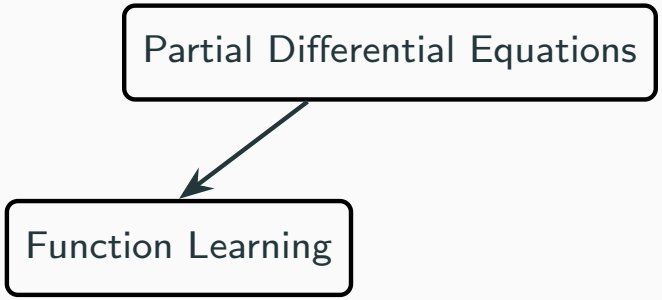
[§] I. Lagaris et al., Neural-network methods for boundary value problems with irregular boundaries, 2000

[§] M. Raissi et al., PINNs: DL for solving forward/inverse problems involving nonlinear PDEs, 2019

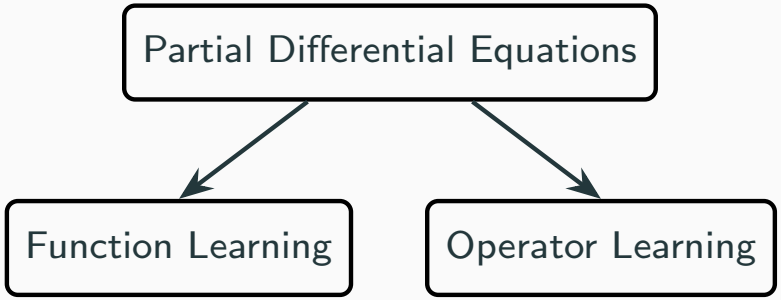


How to embed deeper theoretical insights into scientific machine learning?

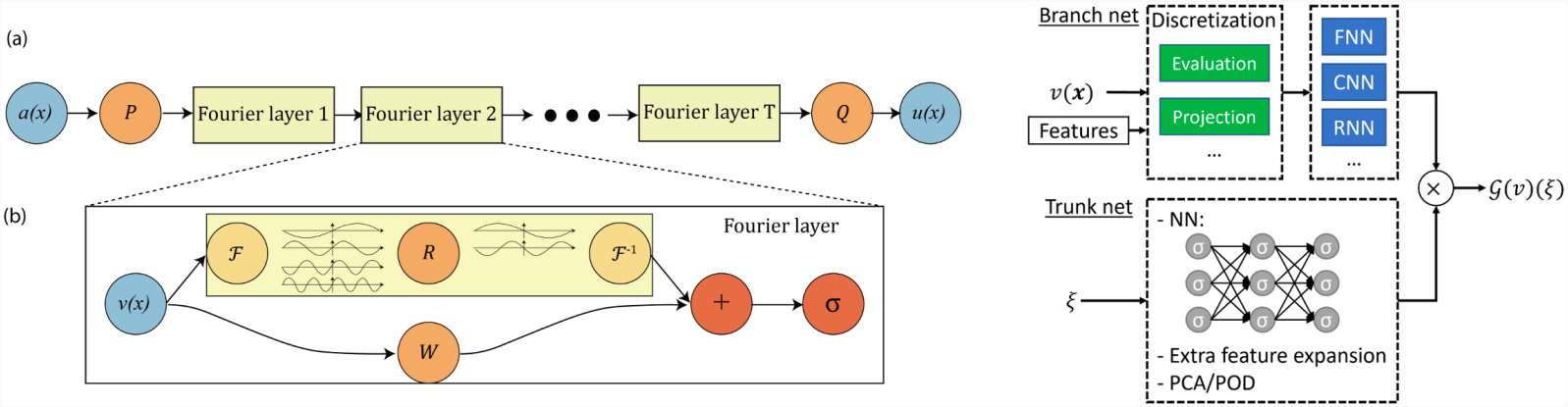
Operator Learning



Operator Learning

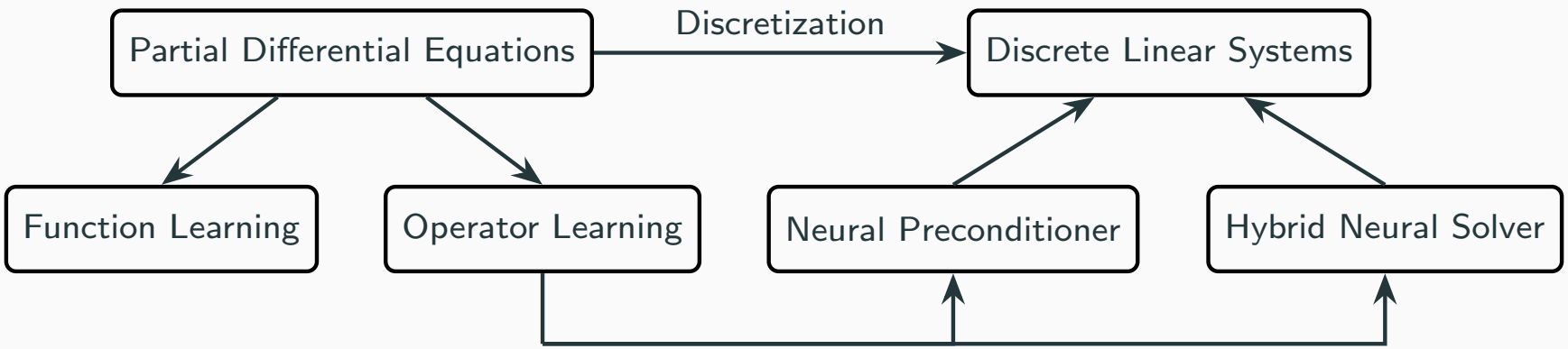


Neural operators learn the solution mapping directly from input-output function pairs, avoiding repeated runs of a numerical solver for new query instances.

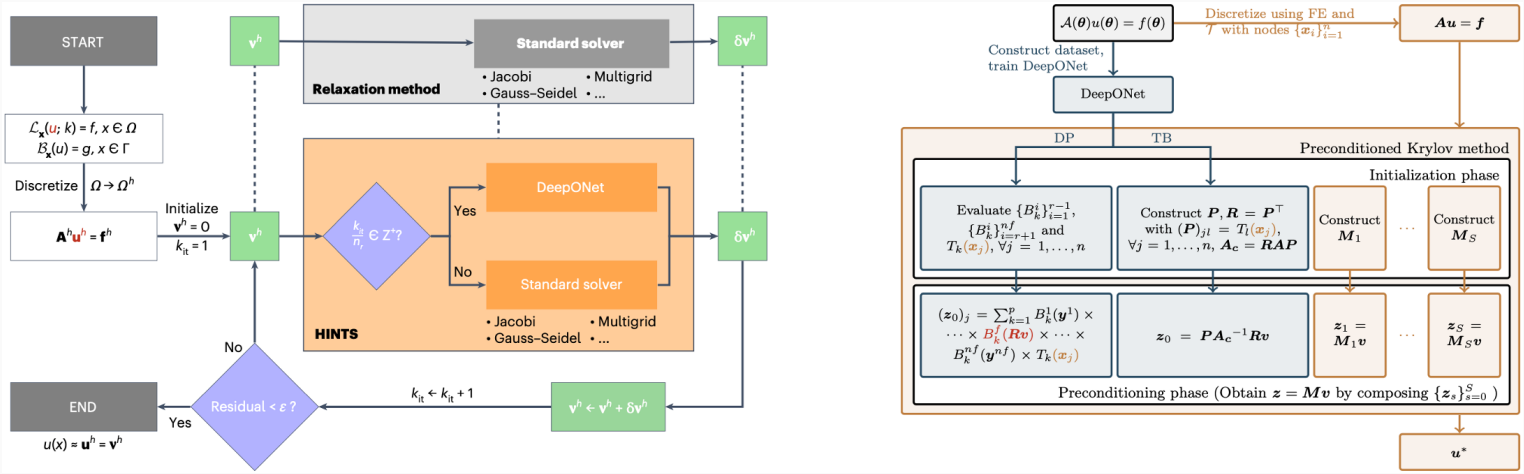


[§] Z. Li et al., Fourier neural operator for parametric partial differential equations, 2020
[§] L. Lu et al., DeepONet: Learning nonlinear operators for identifying differential equations, 2019
[§] T. Chen et al, Universal approximation to nonlinear operators by neural networks, 1995

Operator Learning



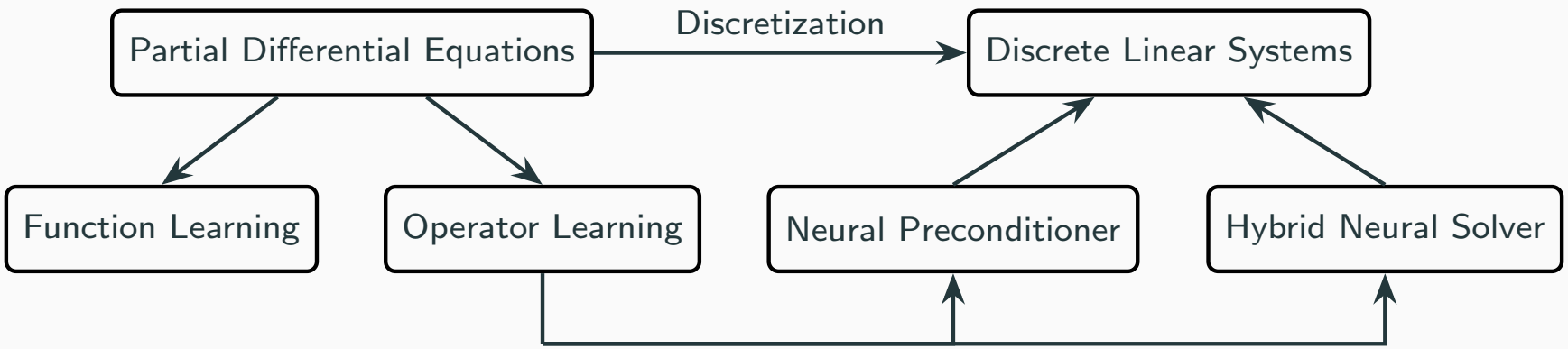
Neural operators learn the solution mapping directly from input-output function pairs, avoiding repeated runs of a numerical solver for new query instances.



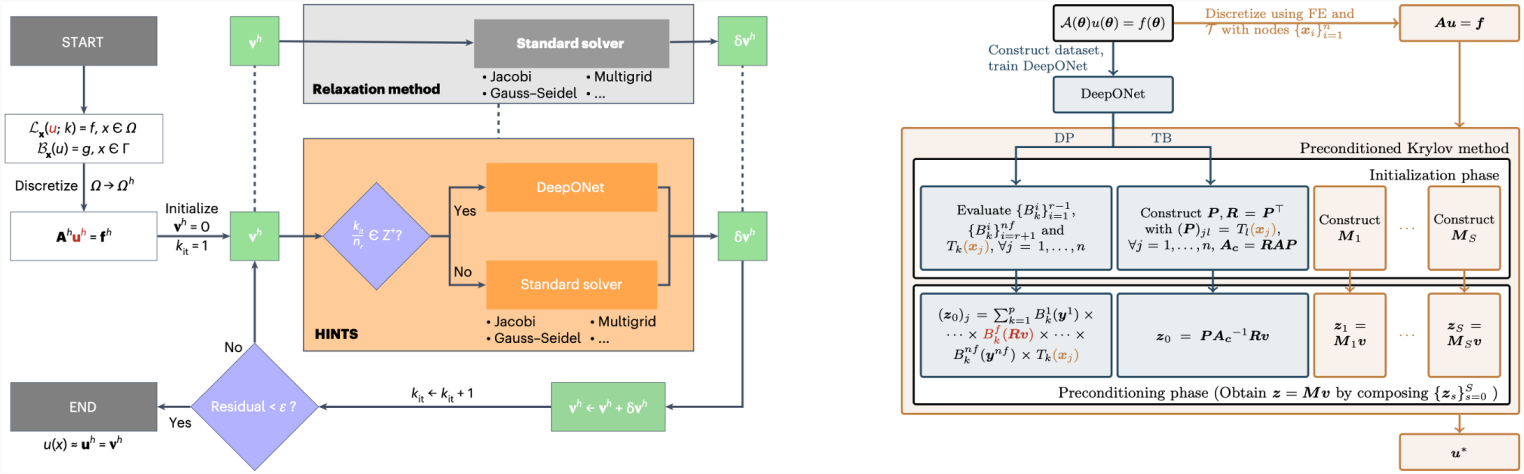
[§] E. Zhang et al., Blending neural operators and relaxation methods in PDE numerical solvers, 2024

[§] A. Kopaničáková et al., DeepONet based preconditioning strategies for parametric linear systems, 2025

Operator Learning



Neural operators learn the solution mapping directly from input-output function pairs, avoiding repeated runs of a numerical solver for new query instances.



[§] E. Zhang et al., Blending neural operators and relaxation methods in PDE numerical solvers, 2024

[§] A. Kopaničáková et al., DeepONet based preconditioning strategies for parametric linear systems, 2025

How to eliminate empirical data dependency and guarantee transparency?

2. Learning Singularity-Encoded Green's Functions

Green's Function

Consider the elliptic boundary value problem under mild assumptions

$$\begin{aligned} -\nabla \cdot (c(x)\nabla u(x)) &= f(x), && \text{in } \Omega \subset \mathbb{R}^d, \\ u(x) &= 0, && \text{on } \partial\Omega, \end{aligned}$$

Green's Function

Consider the elliptic boundary value problem under mild assumptions

$$\begin{aligned} -\nabla \cdot (c(x)\nabla u(x)) &= f(x), \quad && \text{in } \Omega \subset \mathbb{R}^d, \\ u(x) &= 0, && \text{on } \partial\Omega, \end{aligned}$$

where the solution can be expressed through a Hilbert–Schmidt integral operator

$$u(x) = \int_\Omega G(x,y) f(y)\, dy.$$

Here, $G(x,y)$ is the Green's function which satisfies, for any fixed $y \in \Omega$,

$$\begin{aligned} -\nabla \cdot (c(x)\nabla G(x,y)) &= \delta(x-y), \quad && \text{for } x \in \Omega, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega. \end{aligned}$$

[§] L. Evans, Partial differential equations, 2010

[§] L. Stakgold et al., Green's functions and boundary value problems, 2011

[§] Y. Chen et al., Learning based numerical methods for Helmholtz equation with high frequency, 2025

Green's Function

Consider the elliptic boundary value problem under mild assumptions

$$\begin{aligned} -\nabla \cdot (c(x)\nabla u(x)) &= f(x), \quad && \text{in } \Omega \subset \mathbb{R}^d, \\ u(x) &= 0, && \text{on } \partial\Omega, \end{aligned} \xrightarrow{\text{discrete}} AU = F$$

where the solution can be expressed through a Hilbert–Schmidt integral operator

$$u(x) = \int_\Omega G(x,y) f(y)\, dy. \xrightarrow{\text{discrete}} U = A^{-1}F$$

Here, $G(x,y)$ is the Green's function which satisfies, for any fixed $y \in \Omega$,

$$\begin{aligned} -\nabla \cdot (c(x)\nabla G(x,y)) &= \delta(x-y), \quad && \text{for } x \in \Omega, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega. \end{aligned}$$

§ L. Evans, Partial differential equations, 2010

§ L. Stakgold et al., Green's functions and boundary value problems, 2011

§ Y. Chen et al., Learning based numerical methods for Helmholtz equation with high frequency, 2025

Green's Function

Consider the elliptic boundary value problem under mild assumptions

$$\begin{aligned} -\nabla \cdot (c(x)\nabla u(x)) &= f(x), \quad && \text{in } \Omega \subset \mathbb{R}^d, \\ u(x) &= 0, && \text{on } \partial\Omega, \end{aligned} \xrightarrow{\text{discrete}} AU = F$$

where the solution can be expressed through a Hilbert–Schmidt integral operator

$$u(x) = \int_\Omega G(x,y) f(y)\, dy. \xrightarrow{\text{discrete}} U = A^{-1}F$$

Here, $G(x,y)$ is the Green's function which satisfies, for any fixed $y \in \Omega$,

$$\begin{aligned} -\nabla \cdot (c(x)\nabla G(x,y)) &= \delta(x-y), \quad && \text{for } x \in \Omega, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega. \end{aligned}$$

§ L. Evans, Partial differential equations, 2010

§ L. Stakgold et al., Green's functions and boundary value problems, 2011

§ Y. Chen et al., Learning based numerical methods for Helmholtz equation with high frequency, 2025

In the engineering community, Green's functions are also known as

- impulse response (in signal processing);
- influence function (in mechanical engineering);
- point spread function (in optics, astronomical imaging).

§ D. Duffy, Green's functions with applications, 2015

Challenges

While analytical solutions are elegant, they are often limited to idealized scenarios. Numerics are indispensable for realistic problems but face several challenges:

Challenges

While analytical solutions are elegant, they are often limited to idealized scenarios. Numerics are indispensable for realistic problems but face several challenges:

(1) the dimension is doubled, rendering it intractable for mesh-based methods;

$$u(x) : \mathbb{R}^d \to \mathbb{R}, \qquad G(x, y) : \mathbb{R}^{2d} \to \mathbb{R}.$$

Challenges

While analytical solutions are elegant, they are often limited to idealized scenarios. Numerics are indispensable for realistic problems but face several challenges:

(1) the dimension is doubled, rendering it intractable for mesh-based methods;

$$u(x) : \mathbb{R}^d \to \mathbb{R}, \qquad G(x, y) : \mathbb{R}^{2d} \to \mathbb{R}.$$

(2) the computational domain may be very complex or even unbounded;

Challenges

While analytical solutions are elegant, they are often limited to idealized scenarios. Numerics are indispensable for realistic problems but face several challenges:

(1) the dimension is doubled, rendering it intractable for mesh-based methods;

$$u(x) : \mathbb{R}^d \to \mathbb{R}, \qquad G(x, y) : \mathbb{R}^{2d} \to \mathbb{R}.$$

(2) the computational domain may be very complex or even unbounded;

(3) discretization of delta function $\delta(y - x)$ necessitates special treatments;

- approximating with a Gaussian function, but introduces modeling error

$$\rho_\sigma(x, y) = \frac{1}{(\sqrt{2\pi}\sigma)^d} e^{-\frac{\|x-y\|^2}{2\sigma^2}} \xrightarrow{\sigma \to 0} \delta(x, y)$$

[§] Y. Teng et al., Learning Green's functions of linear reaction-diffusion equations, 2021

- employing fundamental solution $G_0(x, y)$, but limited to idealized scenarios

$$G(x, y) = G_0(x, y) + H(x, y) \quad \text{with} \quad \begin{cases} -\nabla \cdot (c(x)\nabla H(x, y)) = 0, & \text{in } \Omega, \\ H(x, y) = -G_0(x, y), & \text{on } \partial\Omega. \end{cases}$$

[§] T. Hu et al., Solving elliptic problems using singularity splitting deep Ritz method, 2023

[§] F. Chen et al., Friedrichs learning: Weak solutions of PDEs via deep learning, 2023

Challenges

While analytical solutions are elegant, they are often limited to idealized scenarios. Numerics are indispensable for realistic problems but face several challenges:

(1) the dimension is doubled, rendering it intractable for mesh-based methods;

$$u(x) : \mathbb{R}^d \to \mathbb{R}, \qquad G(x, y) : \mathbb{R}^{2d} \to \mathbb{R}.$$

(2) the computational domain may be very complex or even unbounded;

(3) discretization of delta function $\delta(y - x)$ necessitates special treatments;

(4) non-trivial approximation of functions with singular behavior near $x = y$;

$$c(x)[\![\nabla G(x, y)]\!] = -1 \ \ (d = 1), \quad G(x, y) \sim \begin{cases} \ln \|x - y\| & (d = 2), \\ \|x - y\|^{2-d} & (d \geq 3). \end{cases}$$

- adopting variational form to transfer derivatives to smooth testing functions

[§] F. Chen et al., Friedrichs learning: Weak solutions of PDEs via deep learning, 2023

- constructing neural network activation functions using rational polynomials

$$\sigma(x) = \frac{p(x)}{q(x)}$$

where $p(x)$ and $q(x)$ are polynomials with trainable coefficients.

[§] N. Boullé et al., Data-driven discovery of Green's functions with interpretable deep learning, 2022

Singularity-Encoded Green's Functions

An augmented variable $\varphi(x, y)$ is included as an extra input into our ansatz, i.e.,

$$G(x, y) = \widehat{G}(x, y, \varphi(x, y)) \quad \text{with} \quad \varphi(x, y) = \begin{cases} |x - y|, & \text{for } d = 1, \\ \ln \|x - y\|, & \text{for } d = 2, \\ \|x - y\|^{2-d}, & \text{for } d \geq 3, \end{cases}$$

Singularity-Encoded Green's Functions

An augmented variable $\varphi(x,y)$ is included as an extra input into our ansatz, i.e.,

$$G(x,y) = \widehat{G}(x,y,\varphi(x,y)) \quad \text{with} \quad \varphi(x,y) = \begin{cases} |x-y|, & \text{for } d=1, \\ \ln\|x-y\|, & \text{for } d=2, \\ \|x-y\|^{2-d}, & \text{for } d \geq 3, \end{cases}$$

and a direct calculation implies

- the continuity of partial derivatives for $d=1$

$$-c(x)[\![\partial_x G(x,y)]\!] = -2c(x)\partial_z \widehat{G}(x,y,\varphi(x,y)) = 1, \text{ for } x=y,$$

- the boundedness of partial derivatives for $d \geq 2$

$$\nabla G(x,y) = \nabla_x \widehat{G}(x,y,\varphi(x,y)) + \nabla\varphi(x,y)\partial_z \widehat{G}(x,y,\varphi(x,y)), \text{ for } x \text{ near } y.$$

Singularity-Encoded Green's Functions

An augmented variable $\varphi(x,y)$ is included as an extra input into our ansatz, i.e.,

$$G(x,y) = \widehat{G}(x,y,\varphi(x,y)) \quad \text{with} \quad \varphi(x,y) = \begin{cases} |x-y|, & \text{for } d=1, \\ \ln\|x-y\|, & \text{for } d=2, \\ \|x-y\|^{2-d}, & \text{for } d \geq 3, \end{cases}$$

and a direct calculation implies

- the continuity of partial derivatives for $d=1$

$$-c(x)[\![\partial_x G(x,y)]\!] = -2c(x)\partial_z \widehat{G}(x,y,\varphi(x,y)) = 1, \text{ for } x = y,$$

- the boundedness of partial derivatives for $d \geq 2$

$$\nabla G(x,y) = \nabla_x \widehat{G}(x,y,\varphi(x,y)) + \nabla\varphi(x,y)\partial_z \widehat{G}(x,y,\varphi(x,y)), \text{ for } x \text{ near } y.$$

Recall equations of the Green's function in terms of normalization condition

$$\begin{aligned} -\nabla\cdot(c(x)\nabla G(x,y)) &= 0, && \text{for } x \in \Omega\setminus\Gamma, \\ -\int_{\partial B_\epsilon(y)} c(x)\nabla G(x,y)\boldsymbol{n}_x \, dS(x) &= 1, && \text{for } x \in \Gamma = \{x \,|\, x = y\}, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega, \end{aligned}$$

for any fixed $y \in \Omega$ and any $\epsilon > 0$, where $B_\epsilon(y) = \{x \,|\, \|x-y\| < \epsilon\}$.

Singularity-Encoded Green's Functions

An augmented variable $\varphi(x,y)$ is included as an extra input into our ansatz, i.e.,

$$G(x,y) = \widehat{G}(x,y,\varphi(x,y)) \quad \text{with} \quad \varphi(x,y) = \begin{cases} |x-y|, & \text{for } d=1, \\ \ln\|x-y\|, & \text{for } d=2, \\ \|x-y\|^{2-d}, & \text{for } d \geq 3, \end{cases}$$

and a direct calculation implies

- the continuity of partial derivatives for $d=1$

$$-c(x)[\![\partial_x G(x,y)]\!] = -2c(x)\partial_z \widehat{G}(x,y,\varphi(x,y)) = 1, \text{ for } x=y,$$

- the boundedness of partial derivatives for $d \geq 2$

$$\nabla G(x,y) = \nabla_x \widehat{G}(x,y,\varphi(x,y)) + \nabla\varphi(x,y)\partial_z \widehat{G}(x,y,\varphi(x,y)), \text{ for } x \text{ near } y.$$

A direct calculation implies our singularity-encoded Green's function satisfies

$$\begin{aligned} \nabla c \cdot (\nabla_x \widehat{G} + \nabla\varphi\partial_z \widehat{G}) + c(\Delta_x \widehat{G} + 2\nabla\varphi \cdot \nabla_x(\partial_z \widehat{G}) + \|\nabla\varphi\|^2 \partial_{zz} \widehat{G}) = 0, & \quad \text{in } \Omega \setminus \Gamma, \\ -\int_{\partial B_\epsilon(y)} c(\nabla_x \widehat{G} + \nabla\varphi\partial_z \widehat{G}) \cdot \boldsymbol{n}_x \, dS(x) = 1, & \quad \text{in } \Gamma, \\ \widehat{G} = 0, & \quad \text{on } \partial\Omega. \end{aligned}$$

for any fixed $y \in \Omega$ and $\epsilon > 0$, where $B_\epsilon(y) = \{x \,|\, \|x-y\| < \epsilon\}$.

[§] Q. Sun et al., Learning singularity-encoded Green's functions with application to iterative methods, 2026

Unsupervised Learning Algorithm

% *Preparation*

– generate training datasets Y_Ω, X_{Reglr}, X_{Bndry}, and X_{Snglr};

– calculate the augmented input $\varphi(x, y)$ for each sample point;

% *Training Process*

– construct and initialize the network model $\widehat{G}(x, y, z; \theta)$;

while maximum number of epochs is not reached **do**

– network training on the shuffled dataset with a suitable learning rate, i.e.,

$$\theta^* = \arg\min_{\theta} \; L_{\text{Reglr}}(\theta) + \beta_{\text{Snglr}} L_{\text{Snglr}}(\theta) + \beta_{\text{Bndry}} L_{\text{Bndry}}(\theta);$$

end while

% *Testing Process*

– forward pass of the trained model on the testing dataset, i.e.,

$$\breve{G}(x, y) = \widehat{G}(x, y, \varphi(x, y); \theta^*).$$

Once trained, the model can be used as a fast solver for arbitrary forcing terms

$$\int_\Omega G(x, y) f(y) dy = u(x) \approx \breve{u}(x) = \sum_{T_p \in \mathcal{T}_h} \sum_{(\omega_y, y_q)} \omega_y f(y_q) \breve{G}(x, y_q).$$

2-1. Numerical Examples

Poisson's Equation in One-Dimension

Consider the Poisson's equation with $u(0) = u(1) = 0$, namely,

$$-u''(x) = f(x), \quad \text{for } x \in \Omega = (0,1),$$

the corresponding Green's function satisfies, for any fixed $y \in (0,1)$,

$$\begin{aligned} -\partial_{xx} G(x,y) &= 0, && \text{for } x \in \Omega \setminus \Gamma, \\ [\![G(x,y)]\!] = 0, \quad [\![\partial_x G(x,y)]\!] &= -1, && \text{for } x \in \Gamma = \{x \,|\, x = y\}, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega, \end{aligned}$$

Poisson's Equation in One-Dimension

Consider the Poisson's equation with $u(0) = u(1) = 0$, namely,

$$-u''(x) = f(x), \quad \text{for } x \in \Omega = (0,1),$$

the corresponding Green's function satisfies, for any fixed $y \in (0,1)$,

$$\begin{aligned} -\partial_{xx} G(x,y) &= 0, && \text{for } x \in \Omega \setminus \Gamma, \\ [\![G(x,y)]\!] = 0, \quad [\![\partial_x G(x,y)]\!] &= -1, && \text{for } x \in \Gamma = \{x \,|\, x = y\}, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega, \end{aligned}$$

while that of singularity-encoded Green's function with $\varphi(x,y) = |x-y|$ reads

$$\begin{aligned} (\partial_{xx} + \partial_{zz})\widehat{G}(x,y,|x-y|) + 2\mathrm{sgn}(x-y)\partial_{xz}\widehat{G}(x,y,|x-y|) &= 0, && \text{in } \Omega \setminus \Gamma, \\ -2\partial_z \widehat{G}(x,y,|x-y|) &= 1, && \text{in } \Gamma, \\ \widehat{G}(x,y,|x-y|) &= 0, && \text{on } \partial\Omega. \end{aligned}$$

Poisson's Equation in One-Dimension

Consider the Poisson's equation with $u(0) = u(1) = 0$, namely,

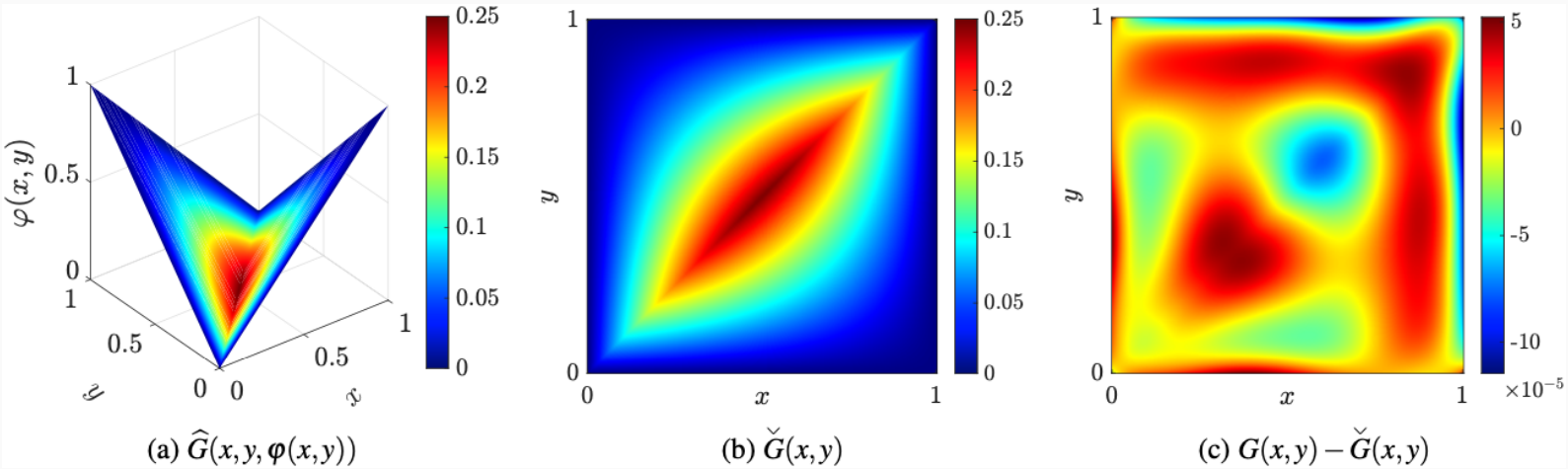
$$-u''(x) = f(x), \quad \text{for } x \in \Omega = (0,1),$$

the corresponding Green's function satisfies, for any fixed $y \in (0,1)$,

$$\begin{aligned} -\partial_{xx} G(x,y) &= 0, && \text{for } x \in \Omega \setminus \Gamma, \\ [\![G(x,y)]\!] = 0, \quad [\![\partial_x G(x,y)]\!] &= -1, && \text{for } x \in \Gamma = \{x \,|\, x = y\}, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega, \end{aligned}$$

while that of singularity-encoded Green's function with $\varphi(x,y) = |x-y|$ reads

$$\begin{aligned} (\partial_{xx} + \partial_{zz})\widehat{G}(x,y,|x-y|) + 2\mathrm{sgn}(x-y)\partial_{xz}\widehat{G}(x,y,|x-y|) &= 0, && \text{in } \Omega \setminus \Gamma, \\ -2\partial_z \widehat{G}(x,y,|x-y|) &= 1, && \text{in } \Gamma, \\ \widehat{G}(x,y,|x-y|) &= 0, && \text{on } \partial\Omega. \end{aligned}$$



(a) $\widehat{G}(x,y,\varphi(x,y))$ (b) $\breve{G}(x,y)$ (c) $G(x,y) - \breve{G}(x,y)$

Poisson's Equation in One-Dimension

Consider the Poisson's equation with $u(0) = u(1) = 0$, namely,

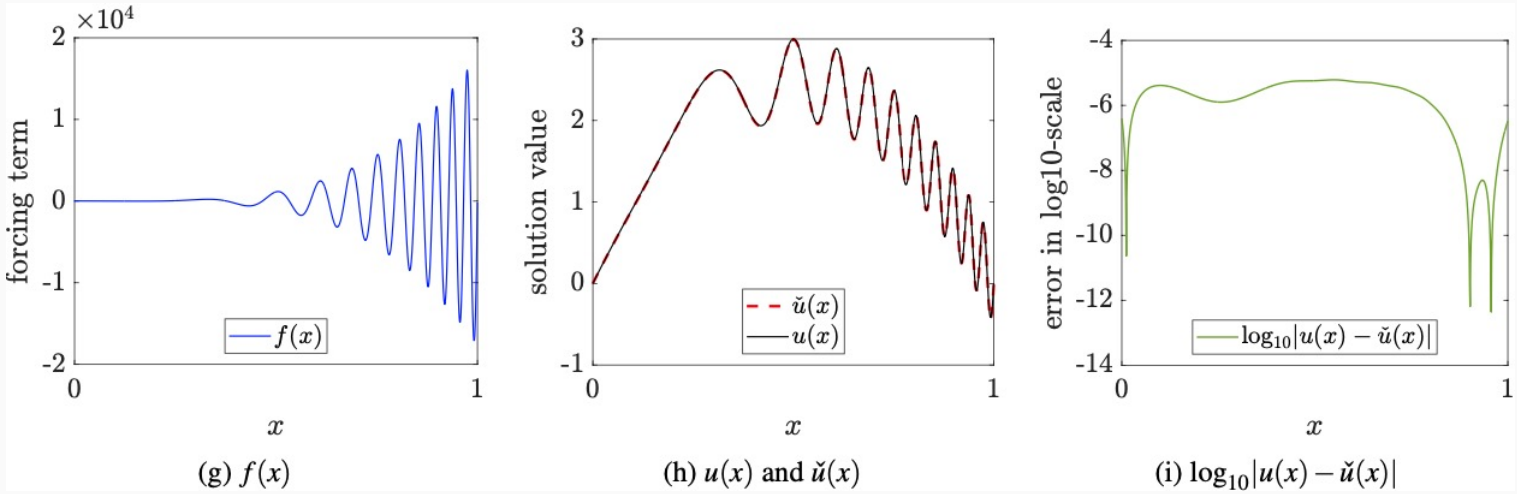
$$-u''(x) = f(x), \quad \text{for } x \in \Omega = (0,1),$$

the corresponding Green's function satisfies, for any fixed $y \in (0,1)$,

$$\begin{aligned} -\partial_{xx} G(x,y) &= 0, && \text{for } x \in \Omega \setminus \Gamma, \\ [\![G(x,y)]\!] = 0, \quad [\![\partial_x G(x,y)]\!] &= -1, && \text{for } x \in \Gamma = \{x \,|\, x = y\}, \\ G(x,y) &= 0, && \text{for } x \in \partial\Omega, \end{aligned}$$

while that of singularity-encoded Green's function with $\varphi(x,y) = |x-y|$ reads

$$\begin{aligned} (\partial_{xx} + \partial_{zz})\widehat{G}(x,y,|x-y|) + 2\mathrm{sgn}(x-y)\partial_{xz}\widehat{G}(x,y,|x-y|) &= 0, && \text{in } \Omega \setminus \Gamma, \\ -2\partial_z \widehat{G}(x,y,|x-y|) &= 1, && \text{in } \Gamma, \\ \widehat{G}(x,y,|x-y|) &= 0, && \text{on } \partial\Omega. \end{aligned}$$

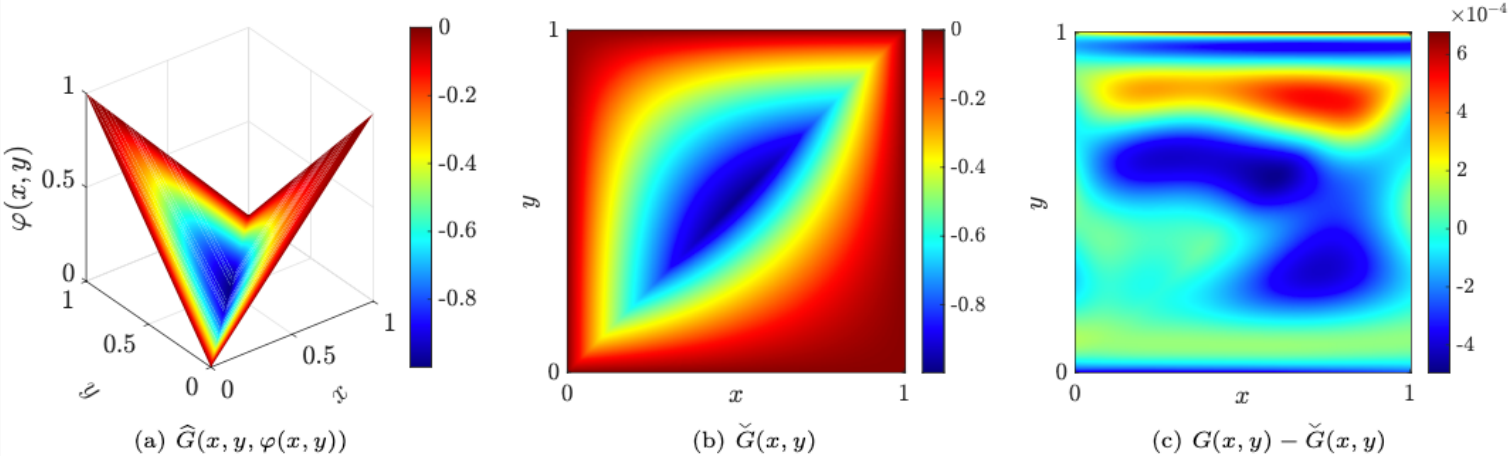


(g) $f(x)$ (h) $u(x)$ and $\check{u}(x)$ (i) $\log_{10}|u(x) - \check{u}(x)|$

Convection-Diffusion/Helmholtz Equations

Numerical results for convection-diffusion equation with $u(0) = u(1) = 0$, i.e.,

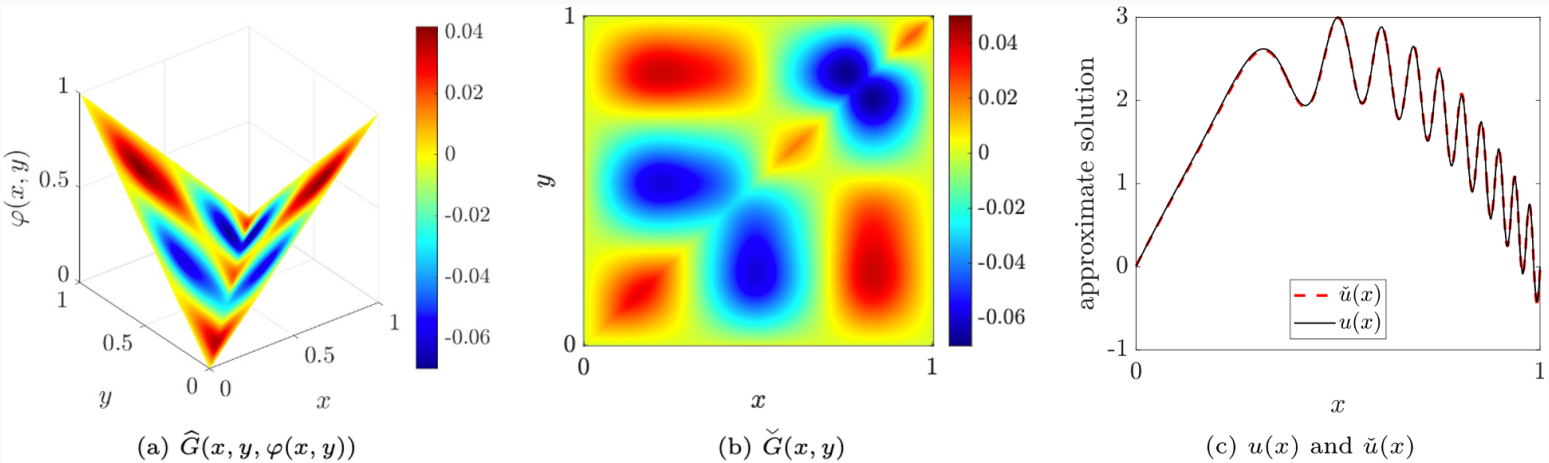
$$\frac{1}{4}u''(x) + \frac{1}{2}u'(x) + \frac{1}{4}u(x) = f(x), \quad \text{for } x \in \Omega = (0,1).$$



(a) $\widehat{G}(x, y, \varphi(x,y))$ (b) $\breve{G}(x,y)$ (c) $G(x,y) - \breve{G}(x,y)$

Numerical results for Helmholtz equation with $u(0) = u(1) = 0$, i.e.,

$$-(c(x)u'(x))' - k^2(x)u(x) = f(x), \quad \text{for } x \in \Omega = (0,1).$$



(a) $\widehat{G}(x, y, \varphi(x,y))$ (b) $\breve{G}(x,y)$ (c) $u(x)$ and $\breve{u}(x)$

Poisson's Equation in Two-Dimension

Consider the two-dimensional Poisson's equation on a unit disc

$$\begin{aligned} -\Delta u(x_1, x_2) &= f(x_1, x_2), \quad && \text{for } (x_1, x_2) \in \Omega = \{(x_1, x_2) \,|\, x_1^2 + x_2^2 \leq 1\}, \\ u(x_1, x_2) &= 0, && \text{for } (x_1, x_2) \in \partial\Omega, \end{aligned}$$

the singularity-encoded Green's function is embedded in a 5-dimensional space

$$G(x_1, x_2, y_1, y_2) = \widehat{G}(x_1, x_2, y_1, y_2, \ln \|\boldsymbol{x} - \boldsymbol{y}\|).$$

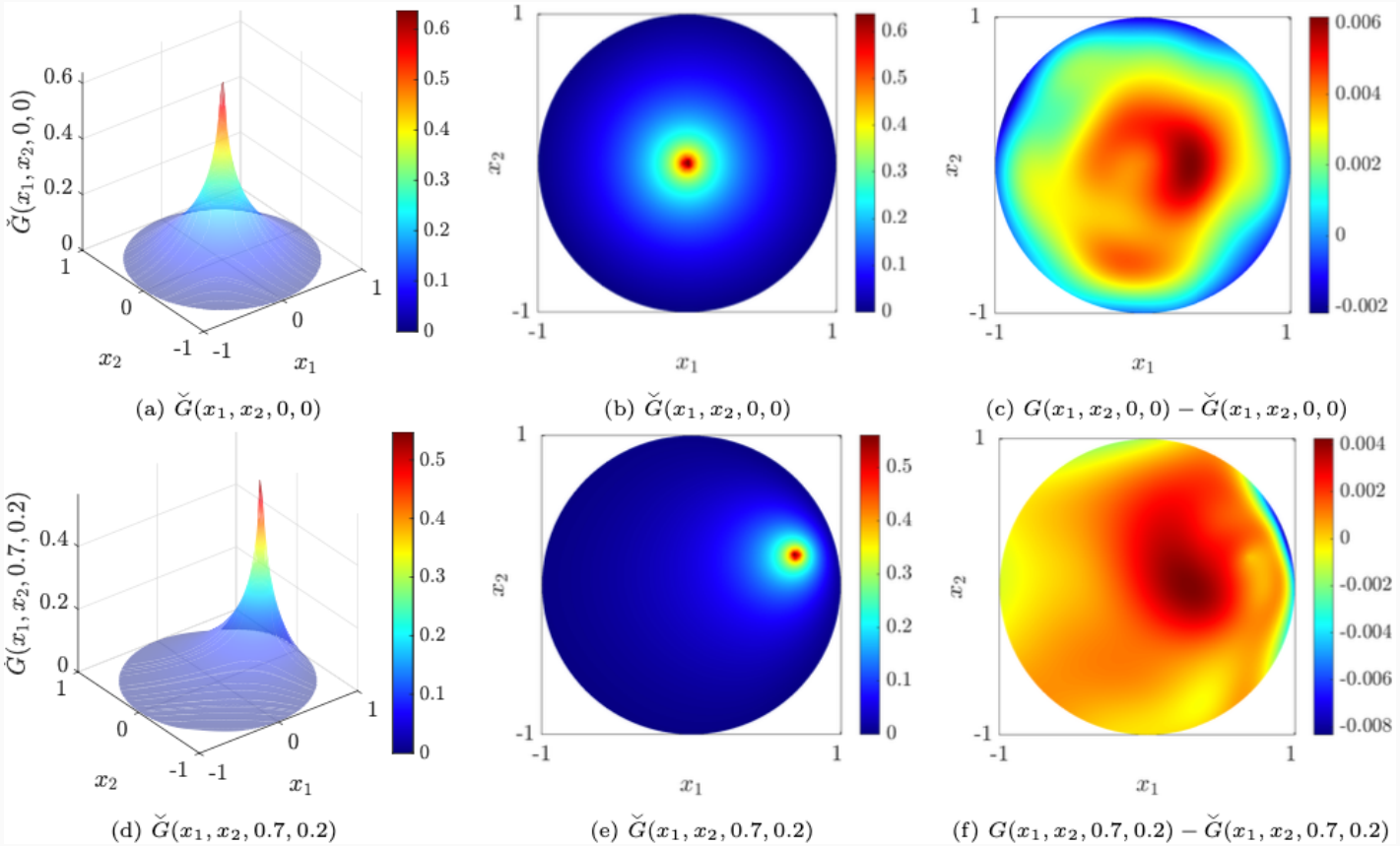
Poisson's Equation in Two-Dimension

Consider the two-dimensional Poisson's equation on a unit disc

$$-\Delta u(x_1, x_2) = f(x_1, x_2), \quad \text{for } (x_1, x_2) \in \Omega = \{(x_1, x_2) \,|\, x_1^2 + x_2^2 \leq 1\},$$
$$u(x_1, x_2) = 0, \quad \text{for } (x_1, x_2) \in \partial\Omega,$$

the singularity-encoded Green's function is embedded in a 5-dimensional space

$$G(x_1, x_2, y_1, y_2) = \widehat{G}(x_1, x_2, y_1, y_2, \ln \|\boldsymbol{x} - \boldsymbol{y}\|).$$



3. Applications in Accelerating Iterative Methods

3-1. Neural Preconditioners

Neural Preconditioner

To accelerate convergence when solving the discretized linear system

$$AU = F$$

a preconditioner $B \approx A^{-1}$ is often applied, leading to the preconditioned system

$$BAU = BF$$

with more clustered eigenvalues while remaining computationally inexpensive.

[§] Q. Sun et al., Learning singularity-encoded Green's functions with application to iterative methods, 2026

Neural Preconditioner

To accelerate convergence when solving the discretized linear system

$$AU = F$$

a preconditioner $B \approx A^{-1}$ is often applied, leading to the preconditioned system

$$BAU = BF$$

with more clustered eigenvalues while remaining computationally inexpensive.

- For small-scale systems, an explicit preconditioning matrix

  $$(\breve{B})_{ij} = \breve{G}(x_i, x_j) \quad \text{for } x_i, x_j \in \mathcal{T}_h,$$

  can be constructed using our singularity-encoded Green's function.

- For large-scale systems, a two-level additive Schwarz preconditioner

  $$\breve{B} = R_0^T \breve{B}_0 R_0 + \sum_{\ell=1}^{L} R_\ell^T A_\ell^{-1} R_\ell \quad \text{with } (\breve{B}_0)_{ij} = \breve{G}(x_i', x_j') \quad \text{for } x_i', x_j' \in \mathcal{T}_H,$$

  where $\mathcal{T}_H$ is the coarse mesh and R_ℓ the matrix of restriction operator on Ω_ℓ.

[§] Q. Sun et al., Learning singularity-encoded Green's functions with application to iterative methods, 2026

Neural Preconditioner

To accelerate convergence when solving the discretized linear system

$$AU = F$$

a preconditioner $B \approx A^{-1}$ is often applied, leading to the preconditioned system

$$BAU = BF$$

with more clustered eigenvalues while remaining computationally inexpensive.

- For small-scale systems, an explicit preconditioning matrix

  $$(\breve{B})_{ij} = \breve{G}(x_i, x_j) \quad \text{for } x_i, x_j \in \mathcal{T}_h,$$

  can be constructed using our singularity-encoded Green's function.

- For large-scale systems, a two-level additive Schwarz preconditioner

  $$\breve{B} = R_0^T \breve{B}_0 R_0 + \sum_{\ell=1}^{L} R_\ell^T A_\ell^{-1} R_\ell \quad \text{with } (\breve{B}_0)_{ij} = \breve{G}(x_i', x_j') \quad \text{for } x_i', x_j' \in \mathcal{T}_H,$$

  where $\mathcal{T}_H$ is the coarse mesh and R_ℓ the matrix of restriction operator on Ω_ℓ.

Consequently, our preconditioned system is formulated as

$$\breve{B}AU = \breve{B}F.$$

[§] Q. Sun et al., Learning singularity-encoded Green's functions with application to iterative methods, 2026

Poisson's Equation in One-Dimension

Consider the discrete linear system of the one-dimensional Poisson's equation

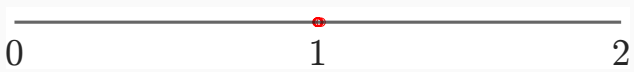
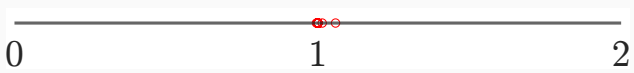
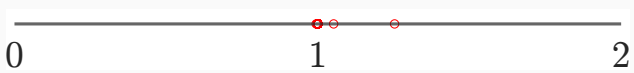
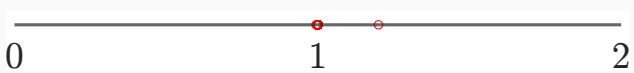
$$A = \frac{1}{h}\text{tri}[-1, 2, -1] \in \mathbb{R}^{n\times n}, \quad U = [U_1, U_2, ..., U_n]^T, \quad F = h[F_1, F_2, ..., F_n]^T,$$

Poisson's Equation in One-Dimension

Consider the discrete linear system of the one-dimensional Poisson's equation

$$A = \frac{1}{h}\text{tri}[-1, 2, -1] \in \mathbb{R}^{n\times n}, \quad U = [U_1, U_2, ..., U_n]^T, \quad F = h[F_1, F_2, ..., F_n]^T,$$

BiCG with/without neural preconditioning for small-scale systems is as follows

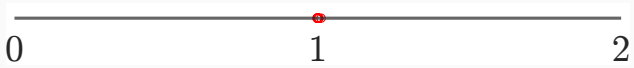
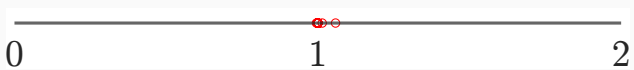
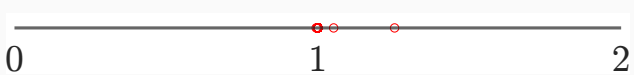
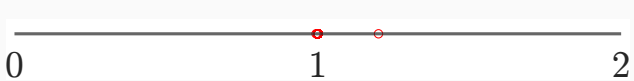
mesh size	eigenvalues of preconditioned matrix	condition number	# iterations	ℓ_2-norm error
$h = 2^{-6}$		$\kappa(\breve{B}A) = 1.01$ $\kappa(A) = 1.66 \times 10^3$	6 (63)	6.54×10^{-15} (8.78×10^{-15})
$h = 2^{-8}$		$\kappa(\breve{B}A) = 1.06$ $\kappa(A) = 2.66 \times 10^4$	6 (255)	3.27×10^{-14} (1.63×10^{-14})
$h = 2^{-10}$		$\kappa(\breve{B}A) = 1.26$ $\kappa(A) = 4.25 \times 10^5$	6 (1023)	7.60×10^{-13} (6.91×10^{-13})
$h = 2^{-12}$		$\kappa(\breve{B}A) = 2.04$ $\kappa(A) = 6.80 \times 10^6$	6 (4095)	4.89×10^{-12} (4.82×10^{-12})

Poisson's Equation in One-Dimension

Consider the discrete linear system of the one-dimensional Poisson's equation

$$A = \frac{1}{h}\text{tri}[-1, 2, -1] \in \mathbb{R}^{n\times n}, \quad U = [U_1, U_2, ..., U_n]^T, \quad F = h[F_1, F_2, ..., F_n]^T,$$

BiCG with/without neural preconditioning for small-scale systems is as follows

mesh size	eigenvalues of preconditioned matrix	condition number	# iterations	ℓ_2-norm error
$h = 2^{-6}$		$\kappa(\breve{B}A) = 1.01$ $\kappa(A) = 1.66 \times 10^3$	6 (63)	6.54×10^{-15} (8.78×10^{-15})
$h = 2^{-8}$		$\kappa(\breve{B}A) = 1.06$ $\kappa(A) = 2.66 \times 10^4$	6 (255)	3.27×10^{-14} (1.63×10^{-14})
$h = 2^{-10}$		$\kappa(\breve{B}A) = 1.26$ $\kappa(A) = 4.25 \times 10^5$	6 (1023)	7.60×10^{-13} (6.91×10^{-13})
$h = 2^{-12}$		$\kappa(\breve{B}A) = 2.04$ $\kappa(A) = 6.80 \times 10^6$	6 (4095)	4.89×10^{-12} (4.82×10^{-12})

while the performance of our additive preconditioner for large-scale systems is

mesh size	coarse grid	overlap	condition number	# iterations	ℓ_2-norm error
$h = 2^{-16}$	$H = 2^{-10}$	$\delta = 30h$	$\kappa(\breve{B}A) = 7.26$	23	7.73×10^{-10}
$h = 2^{-18}$	$H = 2^{-10}$	$\delta = 30h$	$\kappa(\breve{B}A) = 5.51$	23	2.5×10^{-8}

3-2. Hybrid Iterative Methods

Spectral Bias of Classical Iterative Methods

Classical iterative methods can be formulated in a residual-correction form

$$U^{[k+1]} = U^{[k]} + B(F - AU^{[k]}) \quad \text{with } B \approx A^{-1},$$

while the corresponding error propagation $E^{[k]} = U^{[k]} - U$ follows from

$$E^{[k+1]} = (I - BA)E^{[k]}.$$

Spectral Bias of Classical Iterative Methods

Classical iterative methods can be formulated in a residual-correction form

$$U^{[k+1]} = U^{[k]} + B(F - AU^{[k]}) \quad \text{with } B \approx A^{-1},$$

while the corresponding error propagation $E^{[k]} = U^{[k]} - U$ follows from

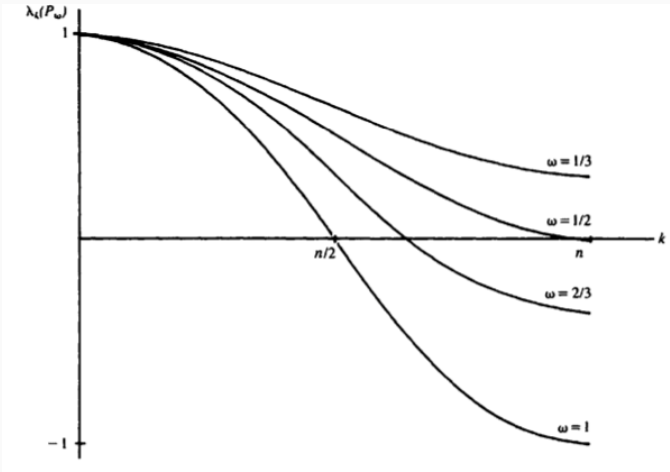
$$E^{[k+1]} = (I - BA)E^{[k]}.$$

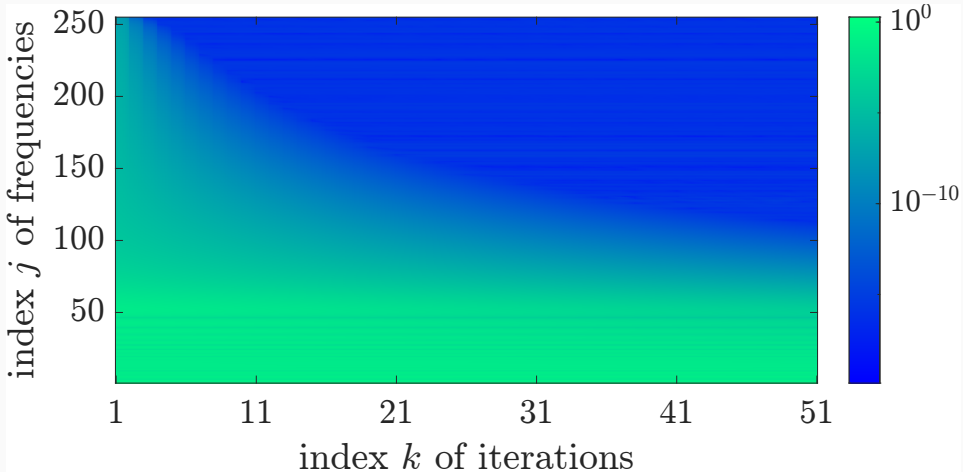
For 1D Poisson's equation, the iterative matrix of damped Jacobi method reads

$$I - BA = (1-\omega)I - \omega D^{-1}(U+L) = I - \frac{\omega}{2}A$$

whose eigenvalue and eigenvector pairs are given by

$$\lambda_j = 1 - 2\omega \sin^2\left(\frac{j\pi}{2n}\right) \quad \text{and} \quad \xi_{j,i} = \sin\left(\frac{ji\pi}{n}\right), \quad 1 \le j \le n-1,\ 0 \le i \le n.$$





Spectral Bias of Neural Preconditioners

By the Mercer's theorem, kernel $\breve{G}(\boldsymbol{x}, \boldsymbol{y})$ admits an L^2-convergent expansion

$$\breve{G}(x, y) = \sum_{j=1}^{\infty} \mu_j \phi_j(x) \phi_j(y)$$

where $\{\mu_j, \phi_j(x)\}_{j \geq 1}$ are eigenpairs of the integral operator

$$\mathcal{K} : L^2(\Omega) \to L^2(\Omega), \quad (\mathcal{K}\phi)(x) := \int_{\Omega} K(x, y)\phi(y)\, dy \quad \text{for any } \phi \in L^2(\Omega).$$

Spectral Bias of Neural Preconditioners

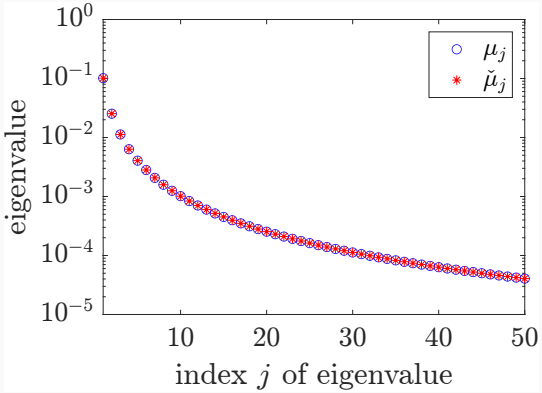
By the Mercer's theorem, kernel $\breve{G}(\boldsymbol{x}, \boldsymbol{y})$ admits an L^2-convergent expansion

$$\breve{G}(x, y) = \sum_{j=1}^{\infty} \mu_j \phi_j(x) \phi_j(y)$$

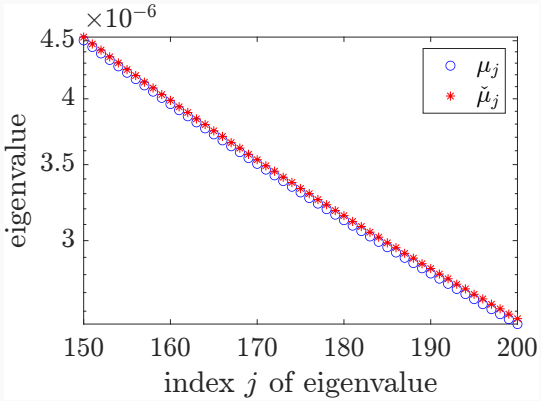
where $\{\mu_j, \phi_j(x)\}_{j \geq 1}$ are eigenpairs of the integral operator

$$\mathcal{K} : L^2(\Omega) \to L^2(\Omega), \quad (\mathcal{K}\phi)(x) := \int_{\Omega} K(x, y)\phi(y)\, dy \quad \text{for any } \phi \in L^2(\Omega).$$

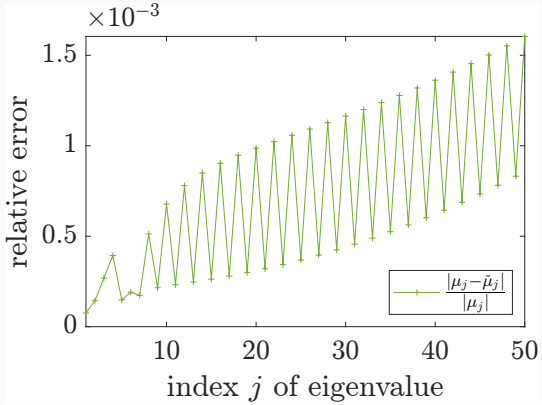
For the Poisson's equation in 1D, our singularity-encoded Green's function exhibits better accuracy in low-frequencies compared to its high-frequency counterparts.



(a) μ_j, $\breve{\mu}_j$ with $1 \leq j \leq 50$



(b) μ_j, $\breve{\mu}_j$ with $151 \leq j \leq 200$



(c) δ_{μ_j} with $1 \leq j \leq 50$

Spectral Bias of Neural Preconditioners

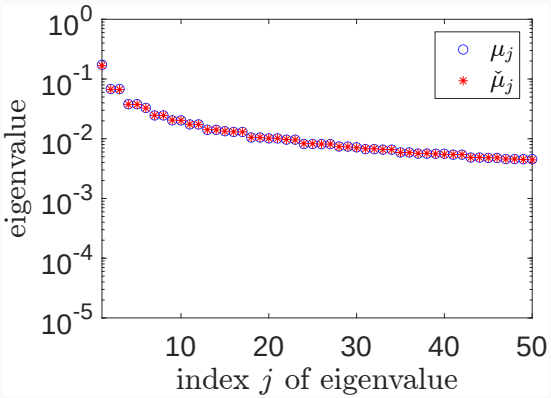
By the Mercer's theorem, kernel $\breve{G}(\boldsymbol{x}, \boldsymbol{y})$ admits an L^2-convergent expansion

$$\breve{G}(x, y) = \sum_{j=1}^{\infty} \mu_j \phi_j(x) \phi_j(y)$$

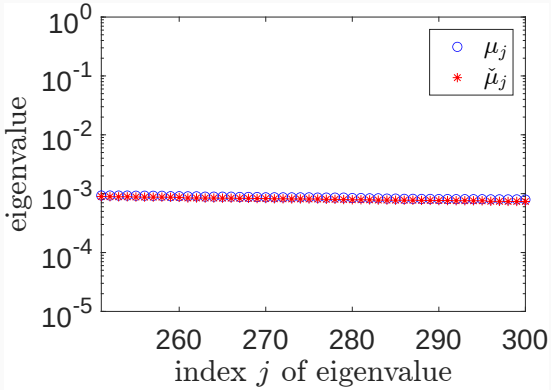
where $\{\mu_j, \phi_j(x)\}_{j \geq 1}$ are eigenpairs of the integral operator

$$\mathcal{K} : L^2(\Omega) \to L^2(\Omega), \quad (\mathcal{K}\phi)(x) := \int_{\Omega} K(x, y)\phi(y)\, dy \quad \text{for any } \phi \in L^2(\Omega).$$

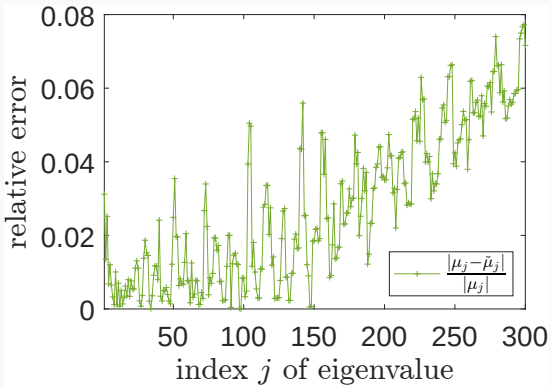
For the Poisson's equation in 2D, our singularity-encoded Green's function exhibits better accuracy in low-frequencies compared to its high-frequency counterparts.



(d) μ_j, $\check{\mu}_j$ with $1 \leq j \leq 50$



(e) δ_{ϕ_j} with $251 \leq j \leq 300$



(f) δ_{μ_j} with $1 \leq j \leq 300$

Spectral Bias of Neural Preconditioners

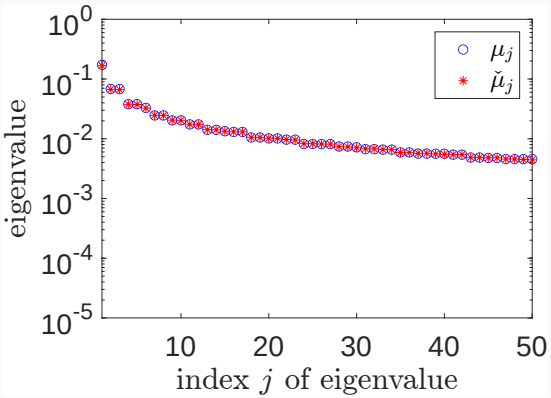
By the Mercer's theorem, kernel $\breve{G}(\boldsymbol{x}, \boldsymbol{y})$ admits an L^2-convergent expansion

$$\breve{G}(x, y) = \sum_{j=1}^{\infty} \mu_j \phi_j(x) \phi_j(y)$$

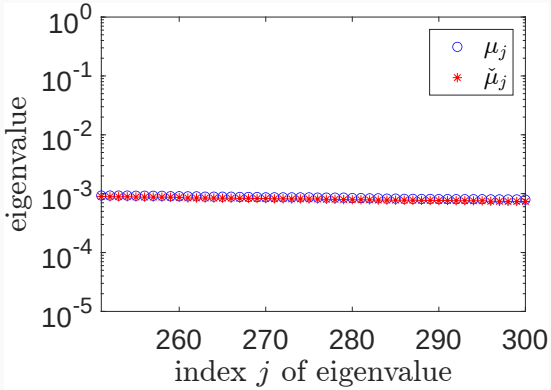
where $\{\mu_j, \phi_j(x)\}_{j \geq 1}$ are eigenpairs of the integral operator

$$\mathcal{K} : L^2(\Omega) \to L^2(\Omega), \quad (\mathcal{K}\phi)(x) := \int_{\Omega} K(x, y)\phi(y)\, dy \quad \text{for any } \phi \in L^2(\Omega).$$

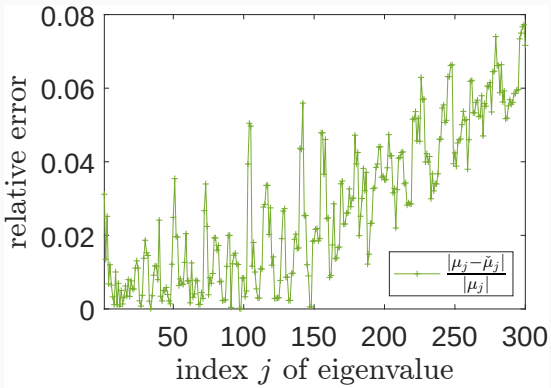
For the Poisson's equation in 2D, our singularity-encoded Green's function exhibits better accuracy in low-frequencies compared to its high-frequency counterparts.



(d) μ_j, $\check{\mu}_j$ with $1 \leq j \leq 50$



(e) δ_{ϕ_j} with $251 \leq j \leq 300$



(f) δ_{μ_j} with $1 \leq j \leq 300$

Neural preconditioners via trained Green's functions suffer from spectral bias!

Hybrid Iterative Methods

Our hybrid iterative methods are formulated in a residual-correction form:

$$U^{[k+1]} = \begin{cases} U^{[k]} + \breve{B}(F - AU^{[k]}), & \text{if } k \equiv 0 \pmod{K}, \\ U^{[k]} + B(F - AU^{[k]}), & \text{otherwise}, \end{cases}$$

while the error propagation follows from

$$E^{[\ell K]} = (I - BA)^{K-1}(I - \breve{B}A)E^{[(\ell-1)K]} = \left[(I - BA)^{K-1}(I - \breve{B}A)\right]^{\ell} E^{[0]}$$

for any $\ell \in \mathbb{N}$, which convergences if and only if the amplification matrix

$$\rho\big((I - BA)^{K-1}(I - \breve{B}A)\big) < 1.$$

Hybrid Iterative Methods

Our hybrid iterative methods are formulated in a residual-correction form:

$$U^{[k+1]} = \begin{cases} U^{[k]} + \breve{B}(F - AU^{[k]}), & \text{if } k \equiv 0 \pmod{K}, \\ U^{[k]} + B(F - AU^{[k]}), & \text{otherwise}, \end{cases}$$

while the error propagation follows from

$$E^{[\ell K]} = (I - BA)^{K-1}(I - \breve{B}A)E^{[(\ell-1)K]} = \left[(I - BA)^{K-1}(I - \breve{B}A)\right]^{\ell} E^{[0]}$$

for any $\ell \in \mathbb{N}$, which convergences if and only if the amplification matrix

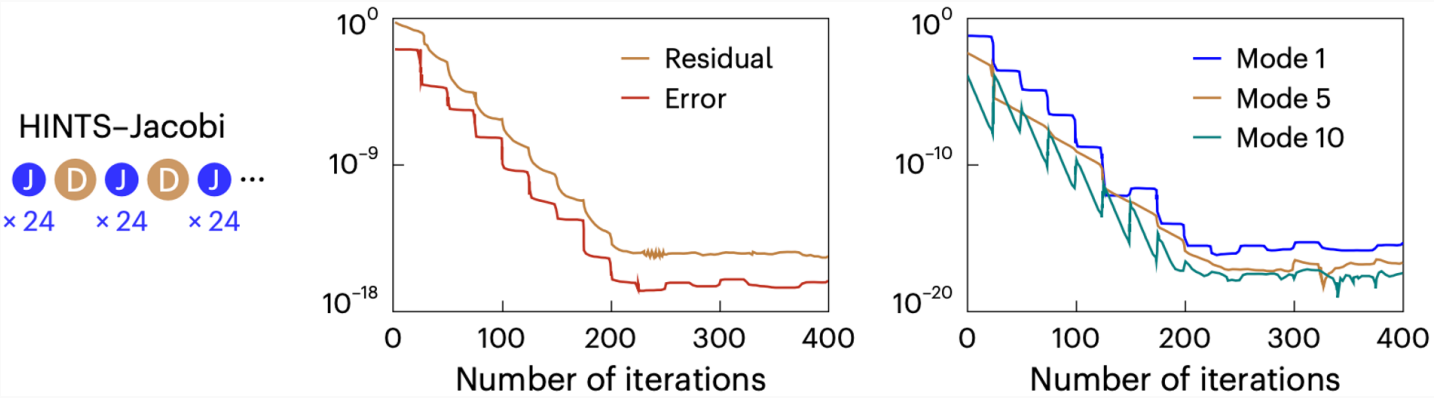
$$\rho\big((I - BA)^{K-1}(I - \breve{B}A)\big) < 1.$$

Thm: *Hybrid iterations converge for sufficiently large K, provided $\rho(I - BA) < 1$.*

[§] E. Zhang et al., Blending neural operators and relaxation methods in PDE numerical solvers, 2024

[§] C. Cui et al., A hybrid iterative neural solver based on spectral analysis for parametric PDEs, 2025

[§] J. Hu et al., A hybrid iterative method based on MIONet for PDEs: Theory and numerical examples, 2025



Hybrid Iterative Methods

Our hybrid iterative methods are formulated in a residual-correction form:

$$U^{[k+1]} = \begin{cases} U^{[k]} + \breve{B}(F - AU^{[k]}), & \text{if } k \equiv 0 \pmod{K}, \\ U^{[k]} + B(F - AU^{[k]}), & \text{otherwise,} \end{cases}$$

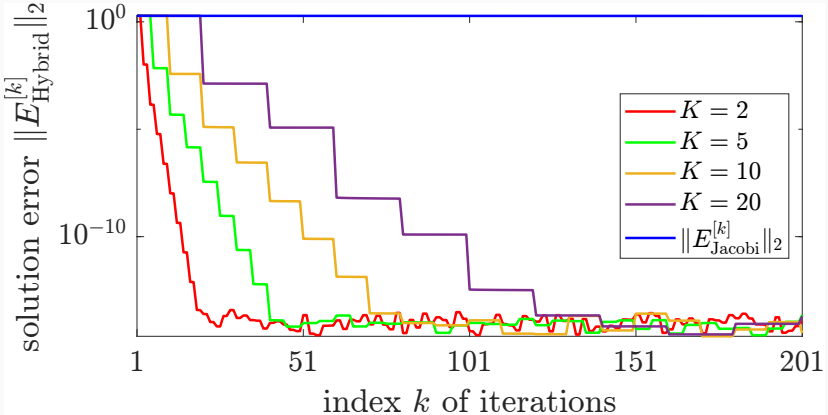
while the error propagation follows from

$$E^{[\ell K]} = (I - BA)^{K-1}(I - \breve{B}A)E^{[(\ell-1)K]} = \left[(I - BA)^{K-1}(I - \breve{B}A)\right]^{\ell} E^{[0]}$$

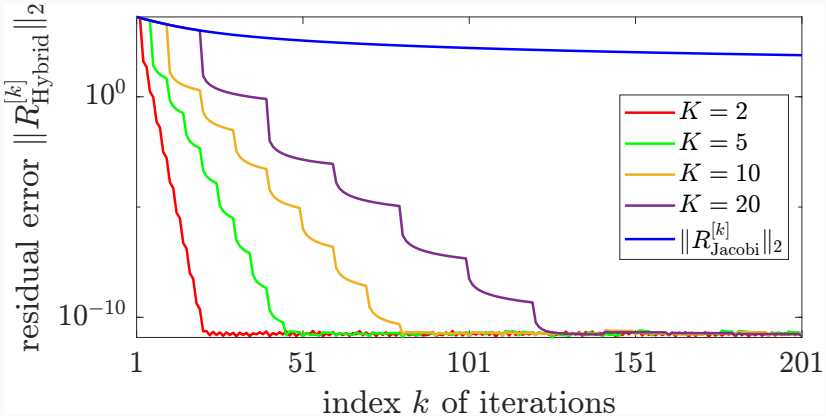
for any $\ell \in \mathbb{N}$, which convergences if and only if the amplification matrix

$$\rho\big((I - BA)^{K-1}(I - \breve{B}A)\big) < 1.$$

Thm: *Hybrid iterations converge for sufficiently large K, provided $\rho(I - BA) < 1$.*



(a) solution errors for the 1D Poisson's equation



(b) residual errors for the 1D Poisson's equation

Hybrid Iterative Methods

Our hybrid iterative methods are formulated in a residual-correction form:

$$U^{[k+1]} = \begin{cases} U^{[k]} + \breve{B}(F - AU^{[k]}), & \text{if } k \equiv 0 \pmod{K}, \\ U^{[k]} + B(F - AU^{[k]}), & \text{otherwise,} \end{cases}$$

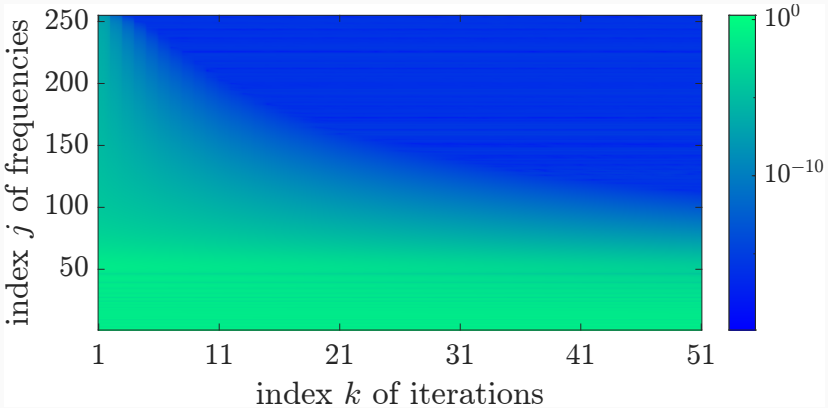
while the error propagation follows from

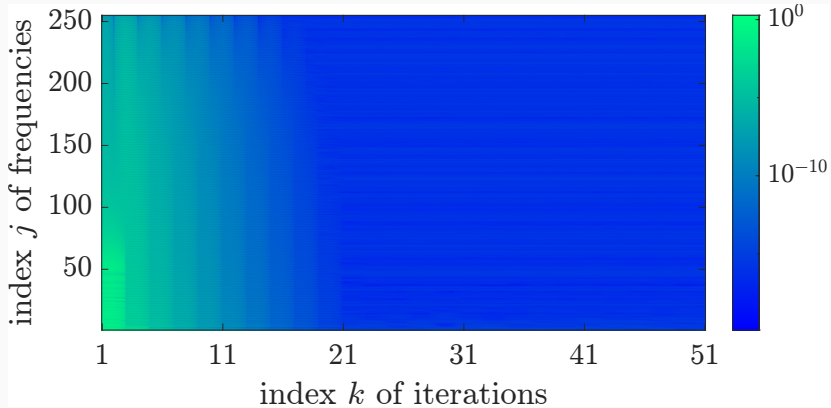
$$E^{[\ell K]} = (I - BA)^{K-1}(I - \breve{B}A)E^{[(\ell-1)K]} = \left[(I - BA)^{K-1}(I - \breve{B}A)\right]^{\ell} E^{[0]}$$

for any $\ell \in \mathbb{N}$, which convergences if and only if the amplification matrix

$$\rho\big((I - BA)^{K-1}(I - \breve{B}A)\big) < 1.$$

Thm: *Hybrid iterations converge for sufficiently large K, provided $\rho(I - BA) < 1$.*





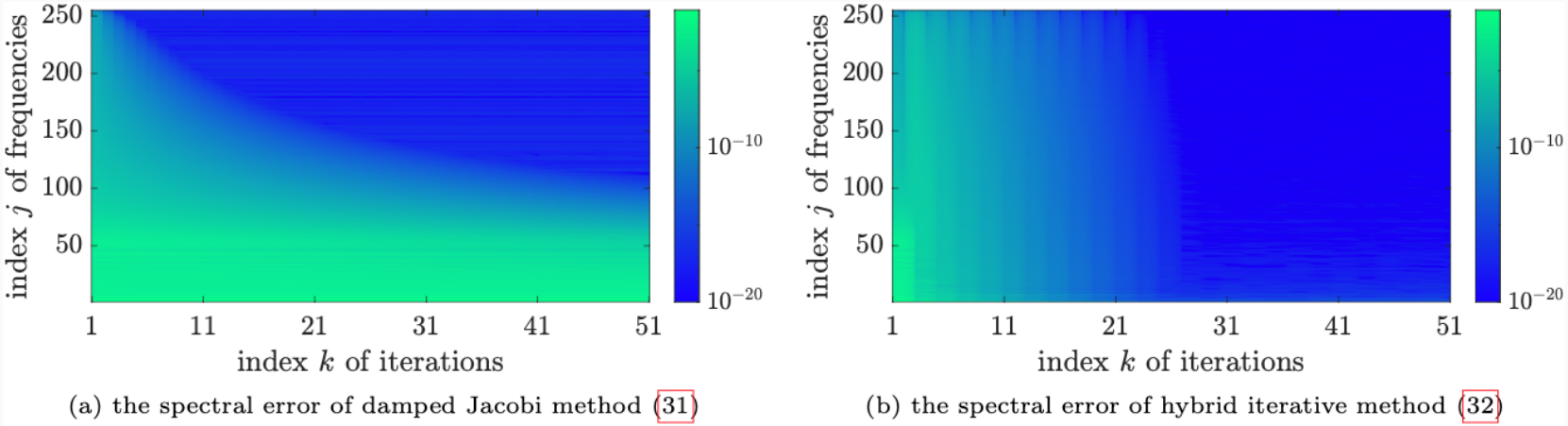
(a) damped Jacobi method for 1D Poisson's equation **(b)** hybrid iterative method for 1D Poisson's equation

Helmholtz Equation in One-Dimension

Recall the Helmholtz equation with variable coefficients, namely,

$$-(c(x)u'(x))' - k^2(x)u(x) = f(x), \quad \text{for } x \in \Omega = (0,1),$$

where $u(0) = u(1) = 0$, the spectral errors of damped Jacobi method and hybrid neural solver applied to its discretized indefinite system are shown below



(a) the spectral error of damped Jacobi method (31)

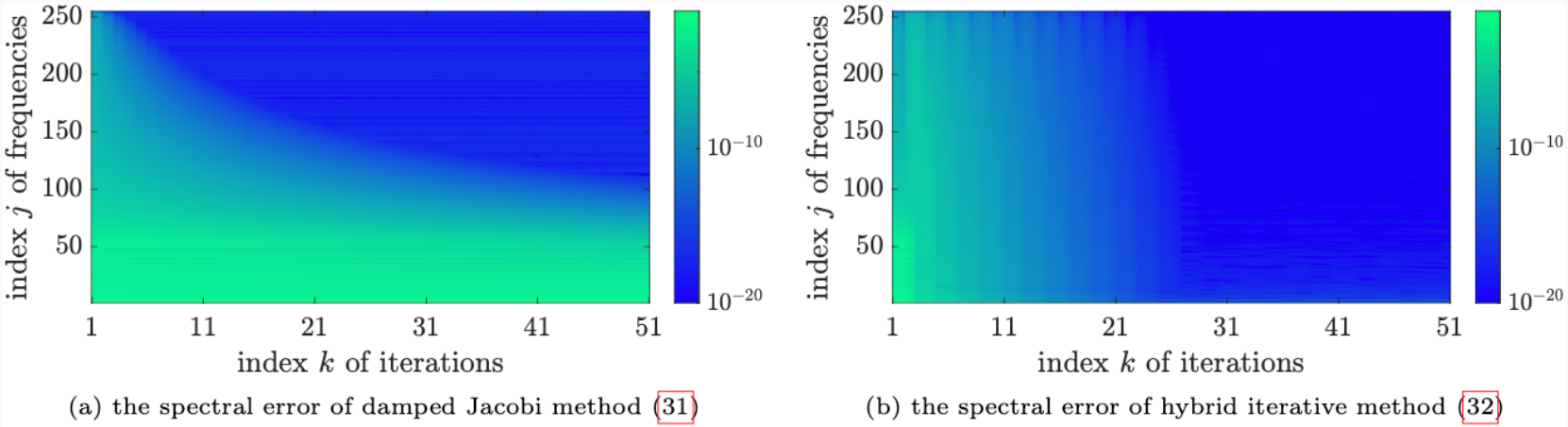
(b) the spectral error of hybrid iterative method (32)

Helmholtz Equation in One-Dimension

Recall the Helmholtz equation with variable coefficients, namely,

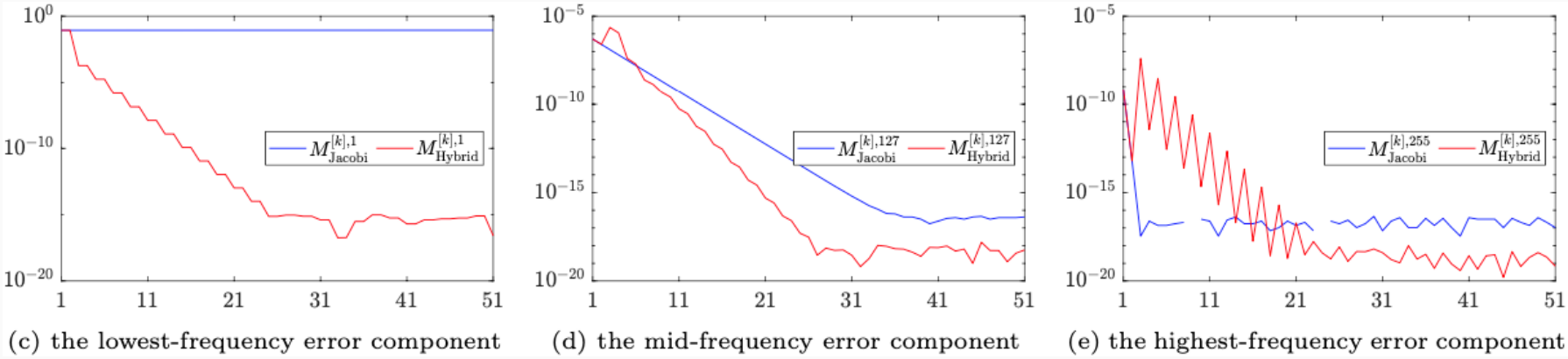
$$-(c(x)u'(x))' - k^2(x)u(x) = f(x), \quad \text{for } x \in \Omega = (0,1),$$

where $u(0) = u(1) = 0$, the spectral errors of damped Jacobi method and hybrid neural solver applied to its discretized indefinite system are shown below



(a) the spectral error of damped Jacobi method (31) (b) the spectral error of hybrid iterative method (32)

followed by its mode-wise errors in the lowest-, mid-, and highest-frequencies



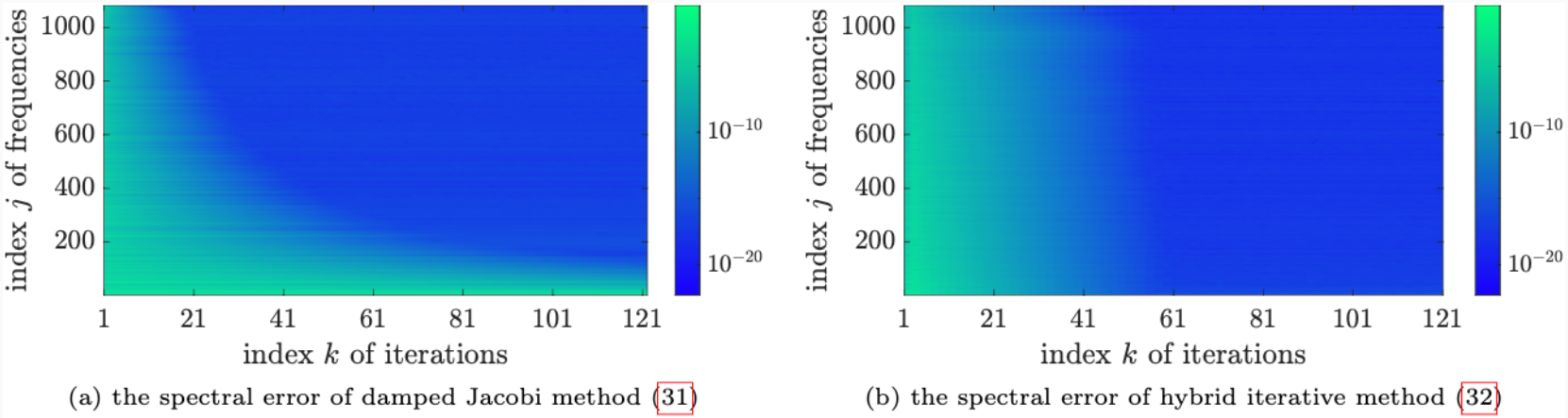
(c) the lowest-frequency error component (d) the mid-frequency error component (e) the highest-frequency error component

Poisson Equation in Two-Dimension

Recall the two-dimensional Poisson's equation on a unit disc

$$-\Delta u(x_1, x_2) = f(x_1, x_2), \quad \text{for } (x_1, x_2) \in \Omega = \{x_1^2 + x_2^2 \leq 1\},$$

where $u(x_1, x_2) = 0$ on $\partial\Omega$, spectral errors of damped Jacobi method and hybrid neural solver applied to its discretized positive-definite system are shown below



(a) the spectral error of damped Jacobi method (31)

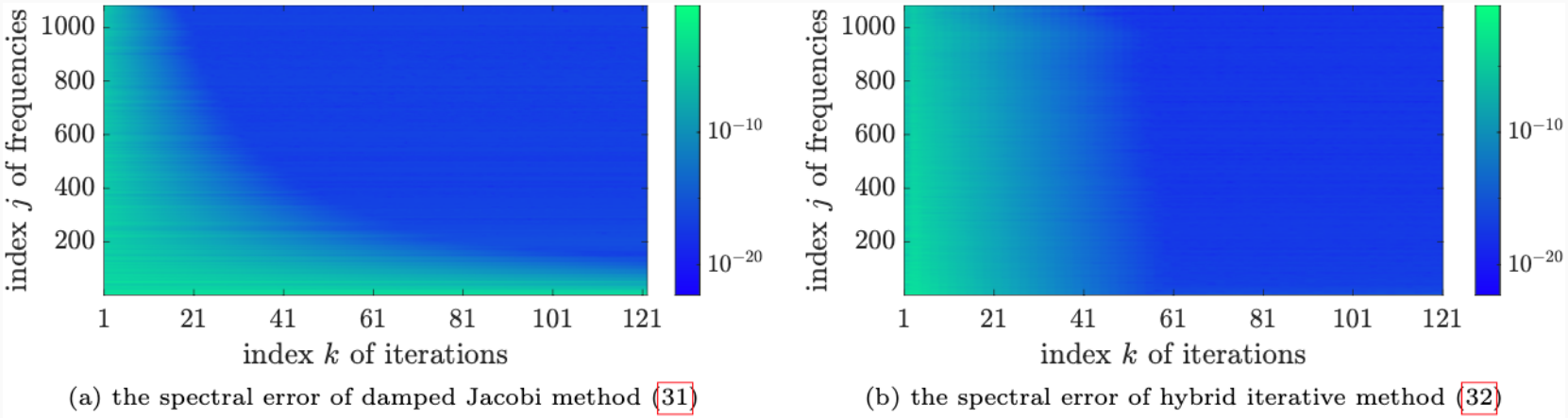
(b) the spectral error of hybrid iterative method (32)

Poisson Equation in Two-Dimension

Recall the two-dimensional Poisson's equation on a unit disc

$$-\Delta u(x_1, x_2) = f(x_1, x_2), \quad \text{for } (x_1, x_2) \in \Omega = \{x_1^2 + x_2^2 \leq 1\},$$

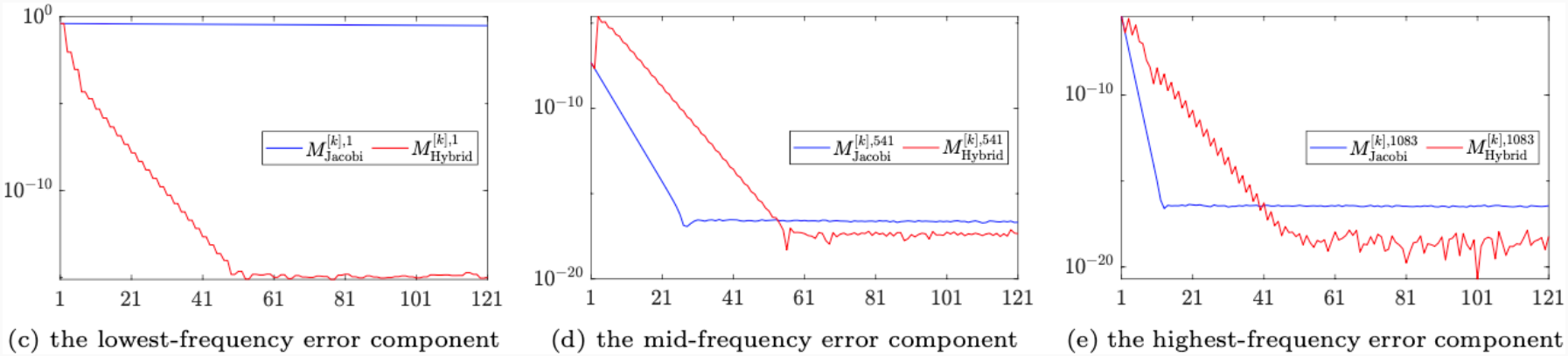
where $u(x_1, x_2) = 0$ on $\partial\Omega$, spectral errors of damped Jacobi method and hybrid neural solver applied to its discretized positive-definite system are shown below



(a) the spectral error of damped Jacobi method (31)

(b) the spectral error of hybrid iterative method (32)

followed by its mode-wise errors in the lowest-, mid-, and highest-frequencies



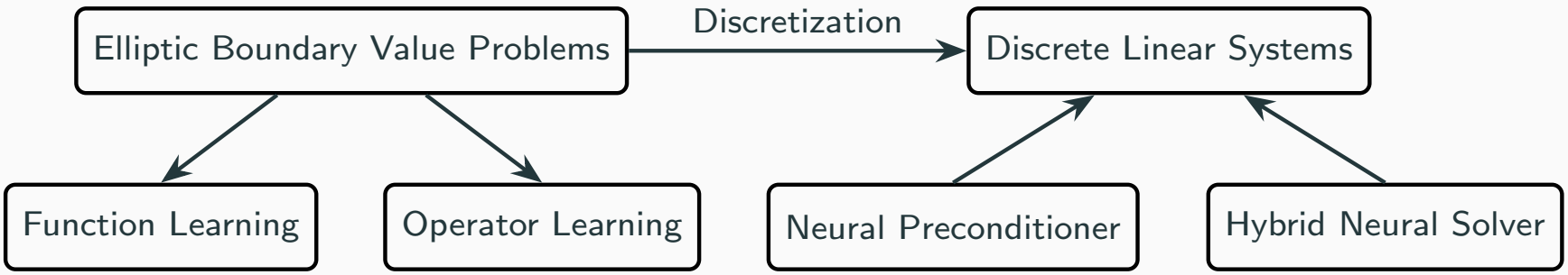
(c) the lowest-frequency error component

(d) the mid-frequency error component

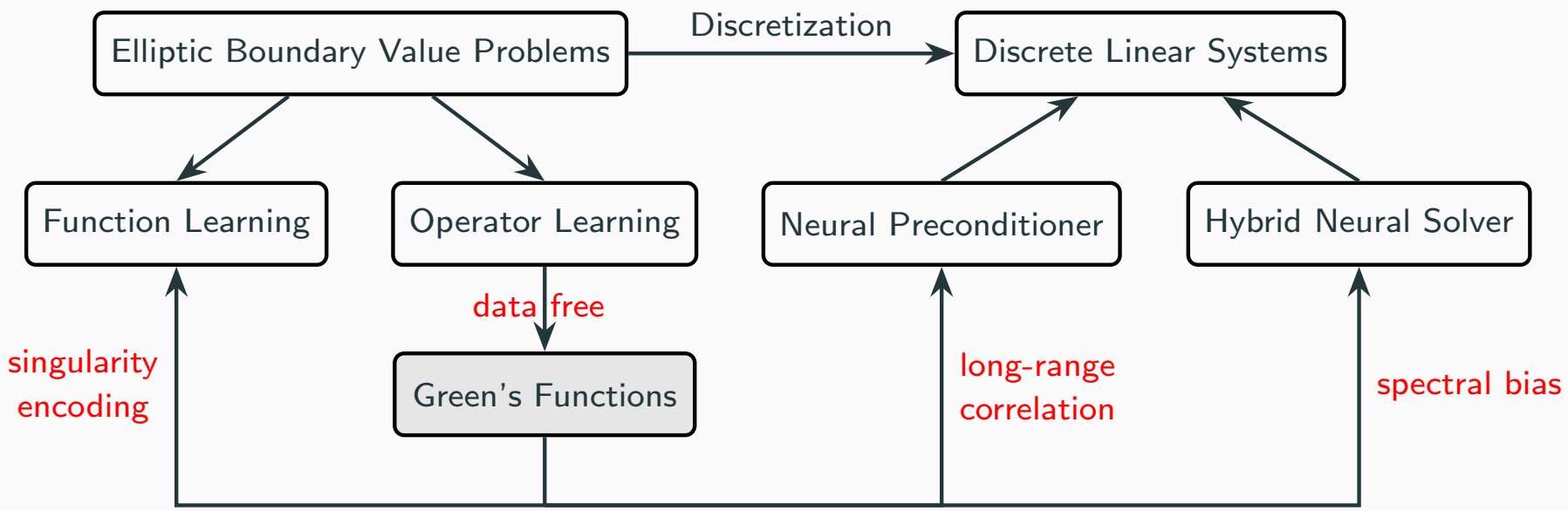
(e) the highest-frequency error component

4. Concluding Remarks

Concluding Remarks

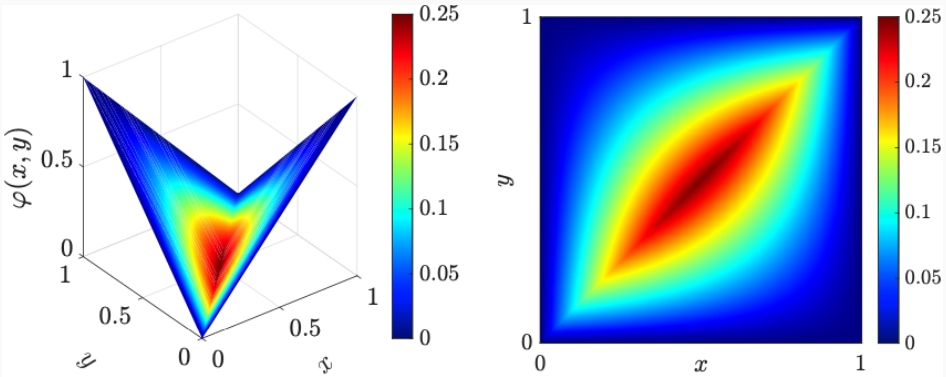


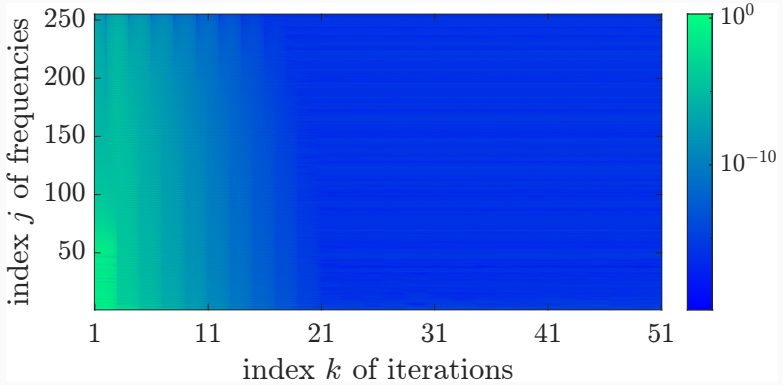
Concluding Remarks



References:

[§] Learning singularity-encoded Green's functions with application to iterative methods
– with S. Li, B. Zheng, L. Ju, and X. Xu, *Journal of Computational Physics*, 2026





Thank You!